

SMART TRAVELOGUE USING FLUTTER

Project Report

Submitted by

JITHA M S

Register No:223242211013

Under the guidance of

Ms.BINNY S

*Submitted in partial fulfilment of the requirement for the award of the degree
of*

MASTER OF COMPUTER APPLICATIONS

AT



DEPARTMENT OF COMPUTER APPLICATION,

KRISTU JYOTI COLLEGE OF MANAGEMENT AND TECHNOLOGY

CHANGANASSERY.

June-2024

KRISTU JYOTI COLLEGE OF MANAGEMENT AND TECHNOLOGY

CHANGANACHERRY

DEPARTMENT OF COMPUTER APPLICATION



CERTIFICATE

This is to certify that the project entitled “**Smart Travelogue using flutter**” has been submitted by **JITHA M S** ,Register Number:**223242211013**, Semester IV,under our guidance and supervision is submitted in partial fulfilment of the Master of Computer Applications, awarded by Mahatma Gandhi University,kottayam, Kerala during theperiod 2022 -2024

Rev.Dr. Joshy Cheeramkuzhy CMI

(Principal)

Ms. Dhannya J

(Project Co-ordinator)

Ms.BINNY S

(Project Guide)

Mr. Roji Thomas

HOD

Submitted for presentation and viva held on.....

Evaluator(s) 1.

2.



CERTIFICATE

This is to certify that **Ms. JITHA M S. (Reg. No: 223242211013)** Master of Computer Applications IV semester student of **KRISTU JYOTI COLLEGE OF MANAGEMENT AND TECHNOLOGY** affiliated to Mahatma Gandhi university, has done project work entitled **"SMART TRAVELOGUE"** in PYTHON+FLUTTER under the guidance of our senior faculties towards the fulfillment of the award of "Master of Computer Applications" during the period of February 2024 to April 2024.

She successfully completed the project and during the period she was methodical and hardworking.

For RISS TECHNOLOGIES

A handwritten signature in green ink, appearing to read "Leborio M", is written over a horizontal line.

Chief Executive Officer



ABSTRACT

Millions of peoples need to be moved from one place to another. For them obtaining a safe journey is a critical issue. Many of them find themselves locked in a city which is not familiar to them by miss the bus, step into the wrong bus, or leave at the wrong station. This is a serious problem faced by the normal peoples especially for a person who is new to the place. The proposed system provides tracking capabilities, low cost, and easy maintenance. Our system helps them for a safe journey by giving the details about the places, important locations , map based on the place etc.

This system provides the details about the hotels and restaurant's near the location we are travelling through , different tour packages available in the hotels, hospitals nearby, other tourist attractions etc. The main attraction of this system include, adding experience videos, audios and travelogues by the people who had already experienced the trip. This will be more useful and interesting for the users who are starting the trip. The introduced system will be a tourist guide for the users in every sense.

DECLARATION

I hereby declare that the project report entitled “**Smart Travelogue using flutter**” submitted to Mahatma Gandhi University in partial fulfillment of requirement for the award of post-graduation of Master of Computer Application from Kristu Jyoti College of Management and Technology, Changanacherry is a record of bonafide work done under the guidance of **Ms.BINNY S** Department of Computer Application. This report has not been submitted in partial or fulfilment of any other post-graduation or similar of this University or any other university.

Place:Changanassery

JITHA M S

Date:

Regno:223242211013

ACKNOWLEDGEMENT

I would like to express my thankfulness towards a number of people who have been instrument in making of this report. First of all, I thank the college management who has been with us withmost helpful facilities throughout the completion of the project.

I am greatly indebted to **Rev. Fr. Joshy Cheeramkuzhy CMI**,Principal of Kristu Jyoti College who whole heartedly give us the permission and whose advice was a real encouragement.

I record my sincere thanks and gratitude to **Mr. Roji Thomas,HOD**,Computer Application,for the valuable suggestions provided.

I am deeply indebted to **Ms.BINNY S Associate. prof.** Department of Computer Application, for thevaluable discussion and helpful counselling.

Last, not the least I wish to express my gratitude and heartfelt thanks to our friends, family and whom with their valuable advice, encouragement and support for the successful completion of the project.

TABLE OF CONTENTS

SL:NO	CONTENTS	PAGES
1	INTRODUCTION	1
2	SYSTEM ANALYSIS	6
3	SYSTEM DESIGN	18
4	SYSTEM ENVIRONMENT	34
5	SYSTEM IMPLEMENTATION	38
6	SYSTEM TESTING	43
7	SYSTEM MAINTENANCE	46
8	SYSTEM SECURITY MEASURES	49
9	SYSTEM PLANNING & SCHEDULING	52
10	SYSTEM COST ESTIMATION	55
11	FUTURE ENHANCEMENT & SCOPE OF FURTHER DEVELOPMENT	58
12	CONCLUSION	65
13	BIBLIOGRAPHY	67
14	APPENDIX	69

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

In today's fast-paced world, the need for efficient and safe travel is paramount, especially for the millions of individuals who find themselves navigating unfamiliar cities. The challenges of missing buses, boarding the wrong ones, or alighting at the wrong stations create significant obstacles for people, particularly those new to a location. Recognizing this pervasive issue, our proposed system aims to revolutionize the travel experience by offering comprehensive tracking capabilities, affordability, and easy maintenance. At the core of our system is a commitment to ensuring a secure journey for individuals by providing essential details about their surroundings. Users can access information about key locations, important landmarks, and interactive maps tailored to their specific locations. This not only facilitates smoother navigation but also alleviates the stress and uncertainty associated with traveling in an unfamiliar city.

1.2 PROBLEM STATEMENT

In the current landscape of mass transportation, a critical issue persists for millions of individuals who need to navigate unfamiliar cities, facing challenges such as missing buses, boarding the wrong ones, or disembarking at incorrect stations. This dilemma is particularly acute for those unfamiliar with a city, leading to potentially serious consequences. The proposed system aims to address these issues by offering tracking capabilities, low-cost solutions, and easy maintenance, ultimately ensuring a safe journey for travelers.

1.3 SCOPE AND RELEVANCE OF THE PROJECT

The scope and relevance of the proposed project are extensive, addressing critical issues faced by millions of individuals who require safe and efficient transportation in unfamiliar cities. The multifaceted nature of the system, encompassing tracking capabilities, affordability, and easy maintenance, positions it as a comprehensive solution with broad implications for users and the tourism industry.

Scope of the Project:

1. Safe Journey Assurance:

- The project focuses on ensuring the safety of individuals during their journeys, mitigating common issues such as missing buses, boarding incorrect ones, or alighting at the wrong stations.

2. Tracking Capabilities:

- Incorporating tracking capabilities enables users to stay informed about their current location, reducing the likelihood of navigation errors and enhancing overall travel security.

3. Cost-Effective and Easy Maintenance:

- By emphasizing low cost and easy maintenance, the project aims to provide an accessible and sustainable solution, making it applicable to a diverse range of users and location

4. Location-Based Information:

- The system offers detailed information about places, important locations, and maps tailored to specific areas, providing users with valuable insights into their surroundings.

Relevance of the Project:

1. User-Centric Approach:

- The project directly addresses the needs of individuals who find themselves in unfamiliar cities, providing them with a user-centric solution that prioritizes safety, convenience, and affordability.

2. Tourism Industry Impact:

- The system's comprehensive approach contributes to the tourism industry by promoting seamless travel experiences, facilitating informed decision-making, and potentially attracting more tourists to destinations with reliable transportation services.

3. Technology-driven Solutions:

- The use of tracking capabilities and multimedia content aligns with contemporary technological trends, offering a modern and innovative solution that caters to the evolving needs of today's travelers.

4. Global Applicability:

- Given the widespread need for efficient transportation in various cities globally, the project's scope and features make it relevant and applicable on a global scale, benefiting diverse populations.

1.4 **OBJECTIVES**

The primary objectives of our proposed system are tailored to address the pressing concerns faced by millions of people reliant on transportation in unfamiliar cities. With safety as a paramount concern, our system aims to provide a secure journey by incorporating tracking capabilities, ensuring that individuals can navigate cities with confidence and avoid common pitfalls such as missing buses or disembarking at the wrong locations. A key focus is on affordability and ease of maintenance, making the system accessible to a broad spectrum of users. In delivering comprehensive location-based information, our objective is to empower users with details about important landmarks, customized maps, and essential locations. Furthermore, our system endeavors to enrich the overall travel experience by offering insights into nearby hotels, restaurants, hospitals, and tourist attractions, along with presenting various tour packages available in hotels. The unique appeal of our system lies in its user-

generated content feature, where experience videos, audios, and travelogues shared by those who have previously undertaken similar journeys serve as a dynamic and engaging guide for users embarking on their trips. By meeting these objectives, our system aspires to not only alleviate the challenges faced by travelers but also emerge as a holistic and interactive tourist guide, catering to the diverse needs of users in every conceivable aspect of their journeys.

CHAPTER 2

SYSTEM ANALYSIS

SYSTEM ANALYSIS

In today's fast-paced world, millions of people travel from one place to another for various reasons. Ensuring a safe and efficient journey for these individuals is of paramount importance. Travelers, especially those unfamiliar with a location, often face challenges such as missing buses, boarding incorrect transport, or alighting at wrong stops. These issues can lead to confusion and stress, particularly for newcomers. The proposed smart travelogue system, developed using Flutter, aims to mitigate these challenges by providing a comprehensive and user-friendly solution.

2.1 EXISTING SYSTEM

As of my last knowledge update in January 2022, there may not be a specific existing system that perfectly mirrors the described proposal. However, various elements of the proposed system align with features present in several travel and navigation applications. Existing systems may incorporate aspects like GPS tracking, location-based services, and information on hotels, restaurants, and tourist attractions.

Common Features in Existing Systems:

1. Navigation Apps:

- Applications like Google Maps, Waze, or other navigation apps provide real-time tracking, route planning, and details about the user's current location.

2. Travel Apps:

- Travel platforms such as TripAdvisor or Yelp offer information about hotels, restaurants, and tourist attractions based on the user's location.

3. Tourism Apps:

- Some cities or regions have their own tourism apps that provide comprehensive information about local attractions, hotels, and transportation options.

4. Experience Sharing Platforms:

- Platforms like YouTube, TripAdvisor, or travel blogs allow users to share their travel experiences through videos, audios, and written travelogues. While these existing systems may cover certain aspects of the proposed system, the uniqueness lies in the integration of all these features into a single, cohesive platform. The proposed system aims to offer a seamless combination of tracking capabilities, cost effectiveness, location-based details, and user-generated content to serve as a comprehensive tourist guide. It's essential to conduct a thorough market analysis and explore the latest developments in travel and navigation applications to identify potential competitors or complementary systems that might have emerged since my last update. Additionally, technological advancements and new startups may have introduced innovative solutions to address the challenges described in the proposal.

➤ **LIMITATIONS OF EXISTING SYSTEM**

There The existing systems in the realm of travel and navigation do offer valuable features, but they also come with certain limitations. These limitations can impact user experience, safety, and overall effectiveness. Here are some common limitations of existing systems:

1. Reliance on Internet Connectivity:

- Many navigation and travel apps heavily depend on a continuous internet connection. This can be a limitation in areas with poor network coverage or when users are traveling internationally and may incur high roaming charges.

2. Incomplete or Outdated Information:

- The details provided by existing systems, such as information about hotels, restaurants, or tourist attractions, may not always be up-to-date. Changes in business operations, closures, or renovations might not be promptly reflected in the application.

3. Limited Integration of User Experiences:

- While some platforms allow users to share experiences through reviews, photos, or comments, the integration of multimedia content like experience videos and audios might be limited. This could restrict the richness and diversity of user-generated content.

4. Privacy Concerns:

- Continuous tracking features in navigation apps may raise privacy concerns among users. The collection and sharing of location data might be perceived as intrusive, leading to reluctance among some users to use certain functionalities.

5. User Interface Complexity:

- Some existing systems might have complex user interfaces that can be challenging for users, especially those who are not tech-savvy or new to the application. A user-friendly design is crucial, particularly for individuals navigating unfamiliar territories.

6. Accessibility Issues:

- The accessibility of existing systems for individuals with disabilities may not be optimal. This limitation can exclude a portion of the population from benefiting fully from the available features.

7. Limited Integration of Health and Safety Information:

- Existing systems may not consistently provide information about nearby hospitals, healthcare facilities, or emergency services, which is crucial for the health and safety of travelers.

8. Dependence on User Reviews:

- While user reviews are valuable, they may vary in quality and reliability. Relying solely on user-generated content might lead to misinformation or biased recommendations.

9. Localization Challenges:

- Existing systems might face challenges in providing accurate information in regions with different languages, dialects, or local nuances, potentially leading to misunderstandings or misinterpretations.

10. Cost Considerations:

- Some navigation apps might have associated costs, particularly for advanced features or premium versions, limiting access for users who cannot afford or are unwilling to pay for such services. Addressing these limitations in the development of a new system can contribute to creating a more comprehensive, user-friendly, and inclusive solution for travelers.

2.2 PROPOSED SYSTEM

The proposed system aims to revolutionize the travel experience for millions of individuals by providing a comprehensive solution that addresses the challenges of navigating unfamiliar cities. Here is an outline of the proposed system:

1. Tracking Capabilities:

- Implement a robust tracking system that allows users to monitor their location in real time. This feature can help users avoid common pitfalls such as missing buses, boarding the wrong vehicles, or leaving at the wrong stations.

2. Cost-Effective and Easy Maintenance:

- Develop the system with a focus on affordability and easy maintenance. This ensures accessibility for a wide range of users and facilitates sustainable implementation in various locations.

3. Location-Based Information:

- Provide users with detailed information about the places they are traveling through, including important landmarks, customized maps based on their specific location, and essential points of interest to enhance their situational awareness.

4. Comprehensive Tourist Services:

- Integrate information about hotels, restaurants, hospitals, and other tourist attractions near the user's location. This feature will serve as a one-stop solution for users, offering a holistic view of the amenities and services available in the vicinity.

5. Tour Packages and Recommendations:

- Highlight different tour packages available in hotels to offer users a variety of options for enhancing their travel experience. These recommendations can be based on user preferences, historical data, and popular attractions.

6. Health and Safety Information:

- Include details about nearby hospitals, healthcare facilities, and emergency services. This ensures that users have access to essential health services during their travels, contributing to their overall safety and well-being.

7. Multimedia Content Integration:

- Allow users to access and contribute experience videos, audios, and travelogues. This dynamic feature enriches the travel planning process, providing valuable insights, tips, and personal anecdotes from individuals who have previously experienced similar journeys.

8. User-Friendly Interface:

- Design an intuitive and user-friendly interface to ensure ease of navigation, especially for individuals who are new to a place or may not be familiar with technology. A visually appealing and straightforward interface contributes to a positive user experience.

9. Offline Functionality:

- Incorporate offline functionality to address issues related to internet connectivity. Users should be able to access essential information, maps, and tracking features even in areas with poor or no network coverage.

10. Global Applicability:

- Ensure that the system is adaptable to various cities and regions globally, considering cultural nuances, language differences, and diverse transportation systems.

11. Continuous Improvement Mechanisms:

- Implement mechanisms for continuous feedback from users. Regular updates based on user experiences, technological advancements, and changing travel trends will enhance the system's effectiveness over time.

ADVANTAGES OF PROPOSED SYSTEM

The proposed system offers several advantages that contribute to a safer, more efficient, and enriching travel experience for individuals. Here are the key advantages of the proposed system:

1. Enhanced Safety:

- The tracking capabilities provide real-time location information, reducing the risk of users getting lost, boarding the wrong transportation, or leaving at the wrong station, thereby ensuring a safer journey.

2. Accessibility and Affordability:

- The focus on low cost and easy maintenance makes the system accessible to a wide range of users, promoting affordability and inclusivity in transportation solutions.

3. Location-Based Information:

- Detailed information about places, landmarks, and customized maps based on the user's location enhance situational awareness, helping users make informed decisions and navigate unfamiliar cities with confidence.

4. Comprehensive Tourist Services:

- Integration of information about hotels, restaurants, hospitals, and tourist attractions provides users with a complete view of the amenities available in the vicinity, making the system a one-stop solution for comprehensive travel planning.

5. Tour Packages and Recommendations:

- Highlighting different tour packages available in hotels adds value to the user's travel experience by offering personalized recommendations and a variety of options for exploration.

6. Health and Safety Information:

- Including details about nearby hospitals and healthcare facilities contributes to the overall safety and well-being of users, ensuring access to essential services during their travels.

7. Multimedia Content Integration:

- The addition of experience videos, audios, and travelogues by fellow travelers enriches the travel planning process, providing users with practical insights, tips, and personal anecdotes, making the journey more interesting and engaging.

8. User-Friendly Interface:

- An intuitive and user-friendly interface ensures that users, even those unfamiliar with technology, can easily navigate the system, promoting a positive user experience.

9. Offline Functionality:

- The inclusion of offline functionality addresses connectivity issues, allowing users to access essential information and tracking features even in areas with poor or no network coverage.

10. Global Applicability:

- The adaptability of the system to various cities and regions globally ensures that users can benefit from its features regardless of cultural differences, language variations, or diverse transportation systems.

2.3 FEASIBILITY STUDY

A feasibility study is essential to assess the viability and practicality of implementing the proposed system. It involves evaluating various aspects, including technical, economic, operational, legal, and scheduling considerations. Here's an overview of the feasibility study for the proposed system:

➤ 2.3.1 Technical Feasibility:

- Hardware and Software Requirements: Assess the technical infrastructure needed for the system, including hardware and software components. Ensure compatibility with a wide range of devices to maximize accessibility.
- Development and Implementation: Evaluate the technical capabilities of the development team and the feasibility of implementing the proposed tracking capabilities, multimedia content integration, and offline functionality.

➤ 2.3.2 Economic Feasibility:

- Cost-Benefit Analysis: Conduct a thorough cost-benefit analysis to determine the economic feasibility of the proposed system. Consider development costs, maintenance expenses, and potential returns on investment.
- Resource Availability: Assess the availability of resources, including finances, skilled personnel, and technology infrastructure, to determine the economic feasibility of the project.

➤ 2.3.3 Operational Feasibility:

- User Adoption: Evaluate the likelihood of user adoption by considering the system's user friendly interface, relevance to the target audience, and potential benefits for individuals seeking safe and convenient travel solutions.
- Operational Impact: Analyze how the proposed system will integrate into existing operations and processes, ensuring minimal disruption and a smooth transition for users.

2.4 SOFTWARE ENGINEERING PARADIGM APPLIED

The proposed system involves various aspects of software engineering, and several software engineering paradigms can be applied to its development. Here are three prominent paradigms that could be considered for implementing the proposed system:

1. Agile Software Development:

- Key Characteristics:
 - Iterative and incremental development.
 - Flexibility to accommodate changes in requirements.
 - Collaboration and communication among cross-functional teams.
- Application to the Proposed System:
 - The agile approach is well-suited for a dynamic project like the proposed system, where user feedback and evolving requirements play a significant role.
 - Regular sprints can be conducted to deliver specific features, allowing for continuous improvements based on user feedback.
 - Collaboration between developers, designers, and content contributors can ensure the integration of multimedia content effectively.

2. DevOps (Development and Operations):

- Key Characteristics:

- Integration of development and operations teams.
- Continuous integration and continuous delivery (CI/CD).
- Automation of software development processes.

- Application to the Proposed System:

- DevOps principles can ensure a seamless integration of tracking capabilities, multimedia content, and real-time information updates.
- Automation of testing and deployment processes can enhance the system's reliability and efficiency.
- Collaboration between developers and operations teams is crucial to maintain the system's performance and address any operational issues promptly.

3. Service-Oriented Architecture (SOA):

- Key Characteristics:

- Decomposition of software into services.
- Interoperability between services through well-defined interfaces.
- Reusability and flexibility in the development of modular components.

- Application to the Proposed System:

- Adopting SOA principles allows for the development of modular components for tracking, location-based information, multimedia content, and other functionalities.
- Well-defined interfaces between services ensure interoperability and ease of integration with third-party systems or data sources.
- Reusable services can be leveraged for similar functionalities across different parts of the system.

Choosing the most suitable paradigm depends on factors such as project complexity, team dynamics, and the desired level of flexibility. In practice, a

combination of these paradigms may be employed to harness the strengths of each and address the specific needs of the proposed system effectively

2.5 FLUTTER

Flutter is an open-source UI software development kit created by Google, used for building natively compiled applications for mobile, web, and desktop platforms from a single codebase. It employs the Dart programming language and provides a rich set of pre-designed widgets for crafting visually appealing and highly performant user interfaces. Flutter's hot reload feature facilitates rapid iteration and experimentation during development, enhancing productivity. Its layered architecture allows for customization and integration with platform-specific features, ensuring a native look and feel across different devices and operating systems.

In terms of types, Flutter primarily encompasses the following:

Mobile Flutter: This is the most common type of Flutter development, where developers use Flutter to build mobile applications for iOS and Android platforms simultaneously.

Web Flutter: Flutter has been extended to support web development, enabling developers to use Flutter to create web applications. With this, developers can target both mobile and web platforms from a single codebase.

Desktop Flutter: Flutter also supports desktop development, allowing developers to create applications for desktop platforms like Windows, macOS, and Linux. This enables the creation of cross-platform desktop applications using Flutter.

Each of these types allows developers to leverage Flutter's capabilities to create high-quality applications across different platforms, providing a unified development experience .

CHAPTER 3

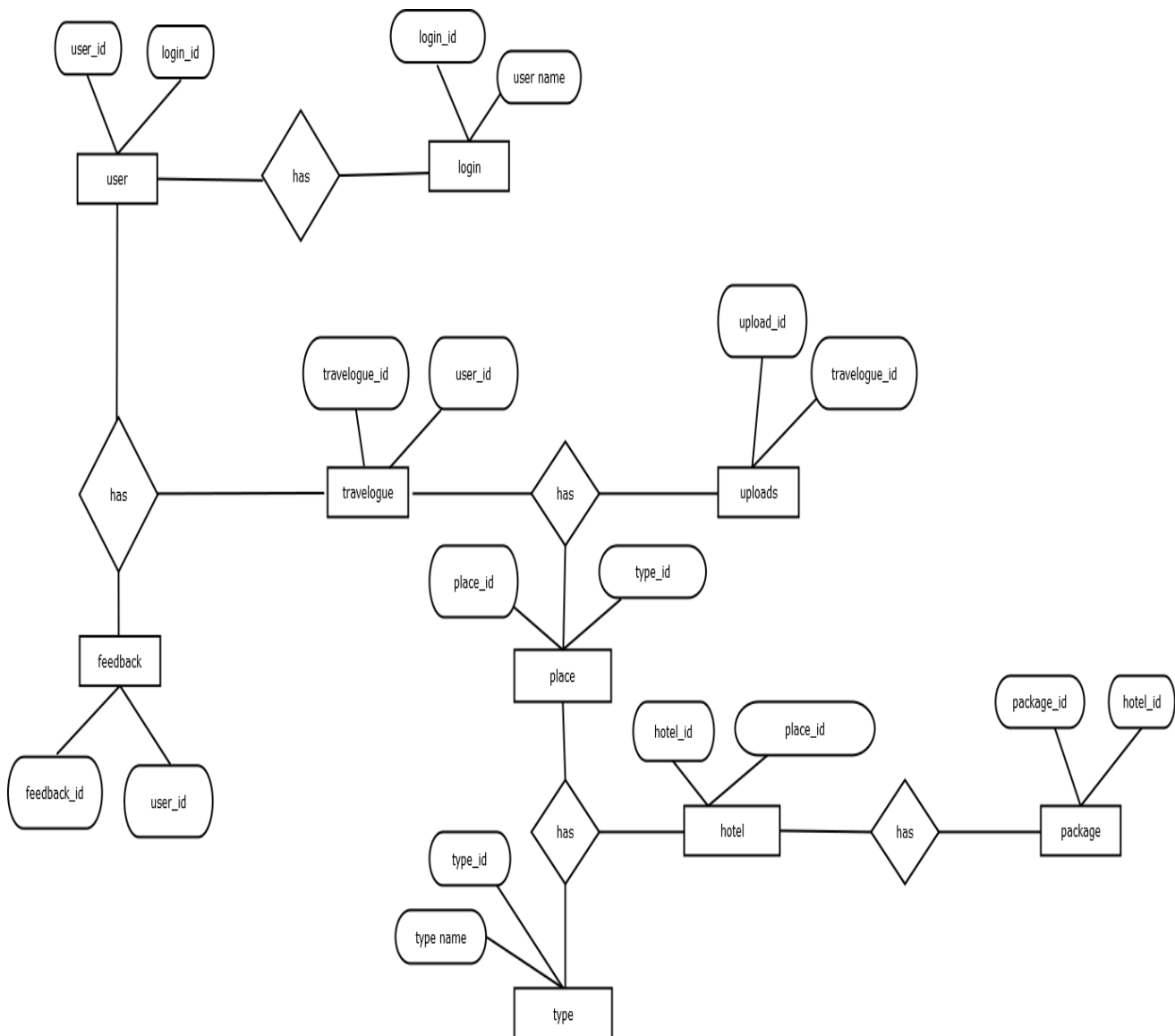
SYSTEM DESIGN

3.1 INTRODUCTION

This chapter outlines the system design for the smart travelogue. It covers the module description, input design and output design, focusing on the main modules: Admin, and user.

3.2 Database Design

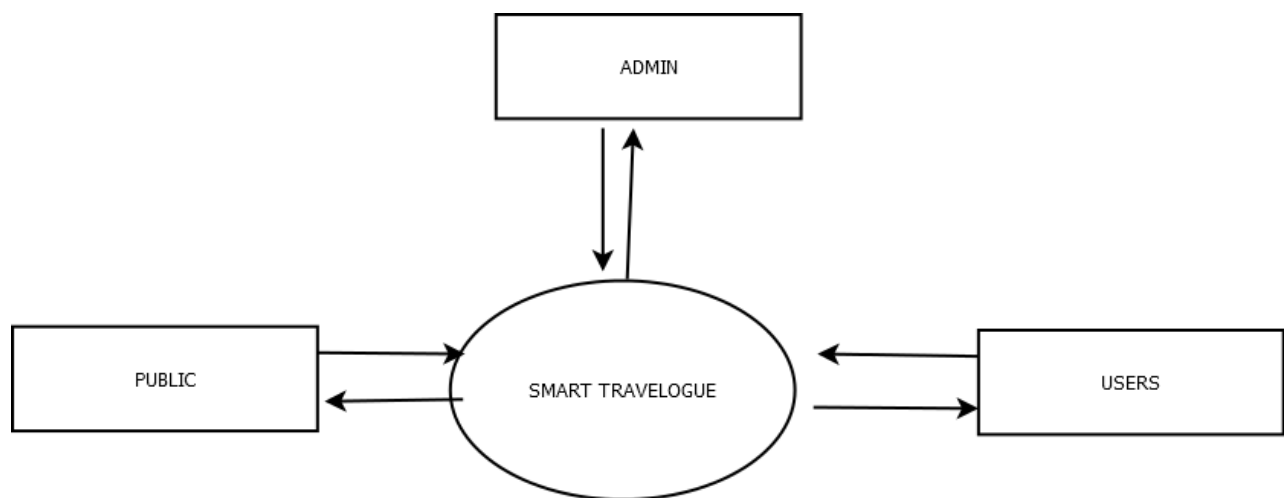
- Entity relationship model



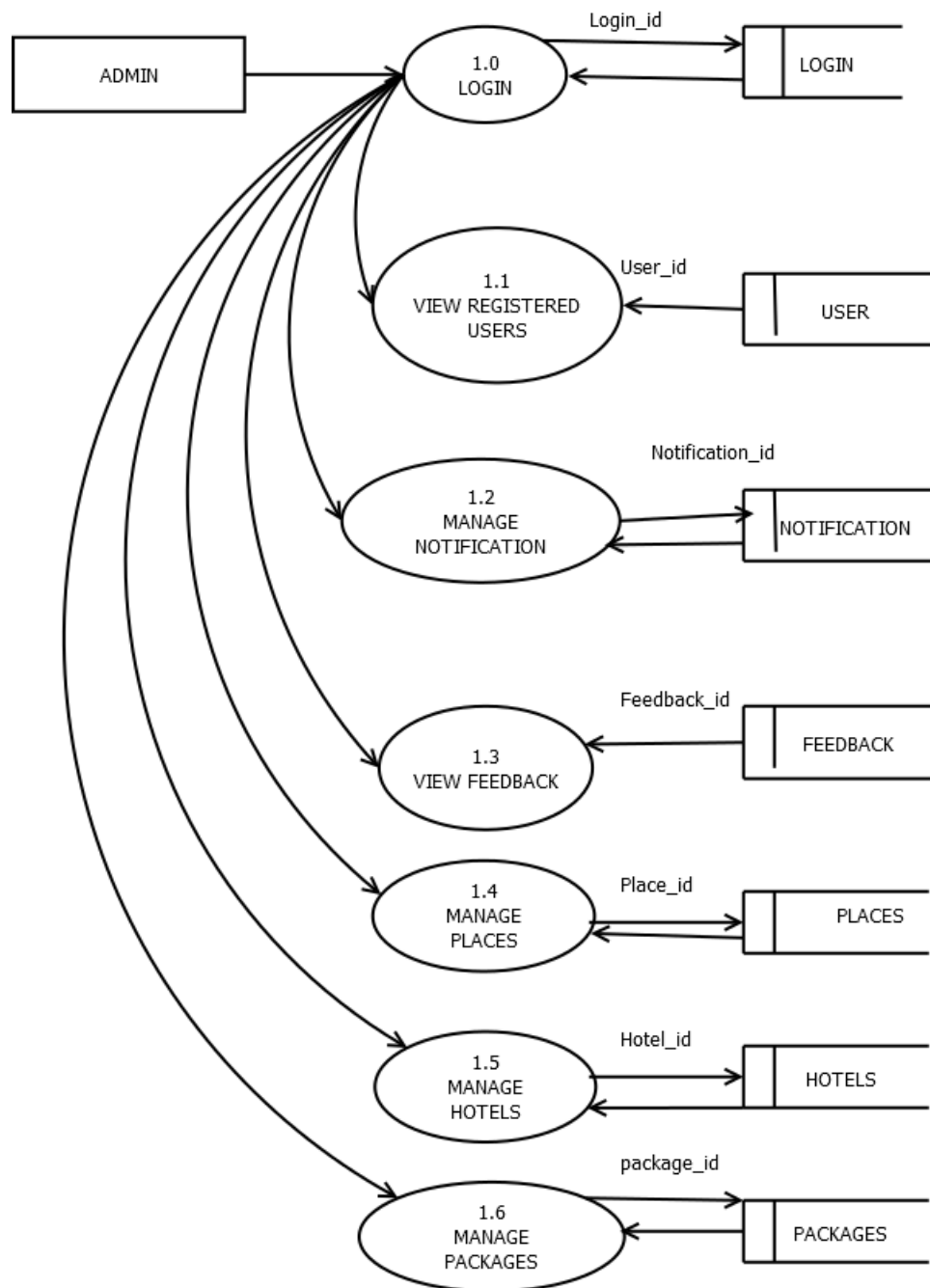
3.3 Process Design-Dataflow diagram

A data-flow diagram (DFD) is a method of illustrating the flow of data within a process or system. It also outlines the inputs and outputs of each component and the process itself. Unlike control flow diagrams, a DFD does not include decision rules or loops. Its purpose is to depict the scope and boundaries of a system comprehensively. Additionally, a DFD can serve as a communication tool between a system analyst and stakeholders, acting as a foundation for system redesign. These diagrams can be applied to analyze systems or software at various levels of abstraction. DFDs can be organized into levels to depict increasing information flow and functional detail, typically numbered as 0, 1, 2, and so forth. The primary levels in a DFD are 0-level, 1-level, and 2-level.

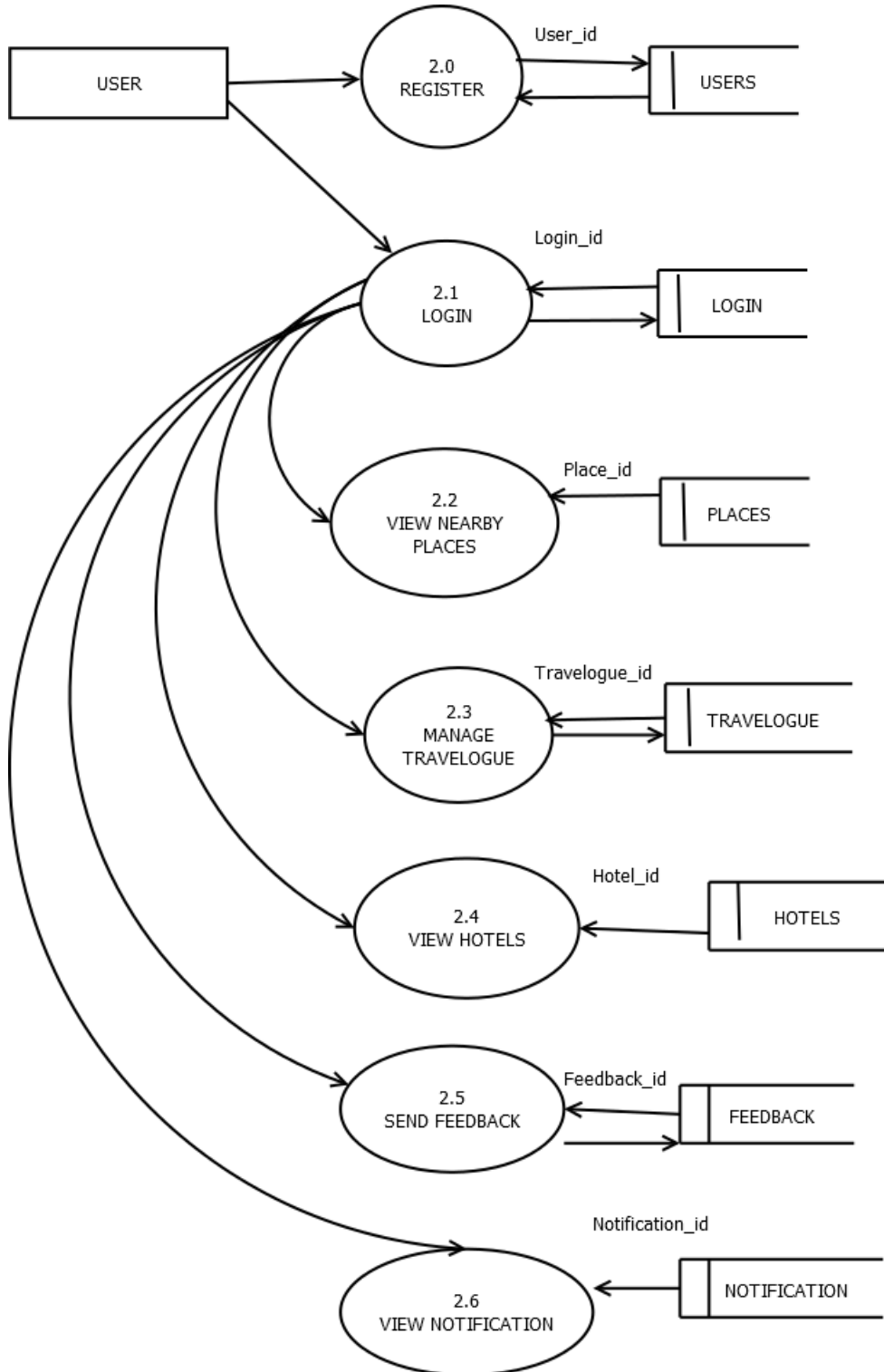
- **Level 0 DFD**



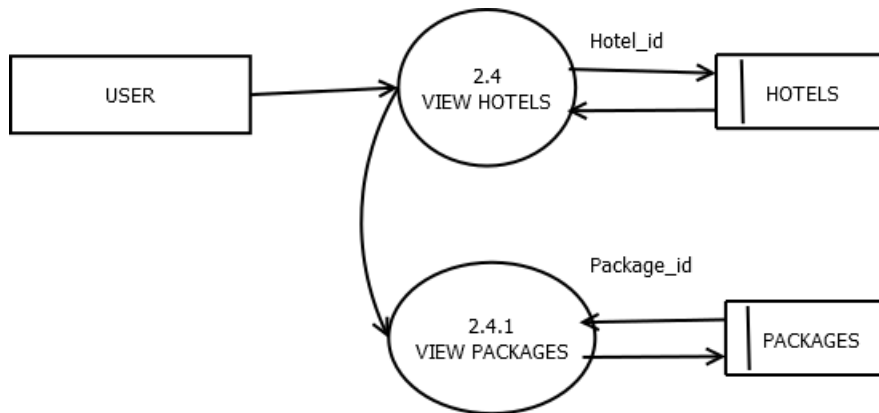
- **Level 1 ADMIN**



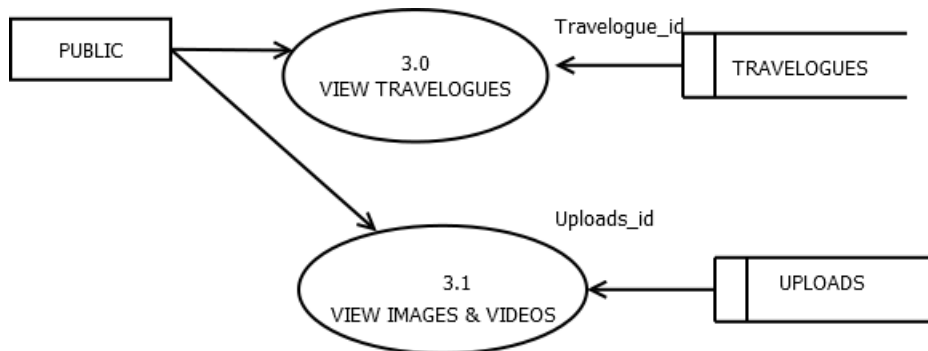
- **Level 1 USER**



- **Level 2 USER**



- **DFD-Public**

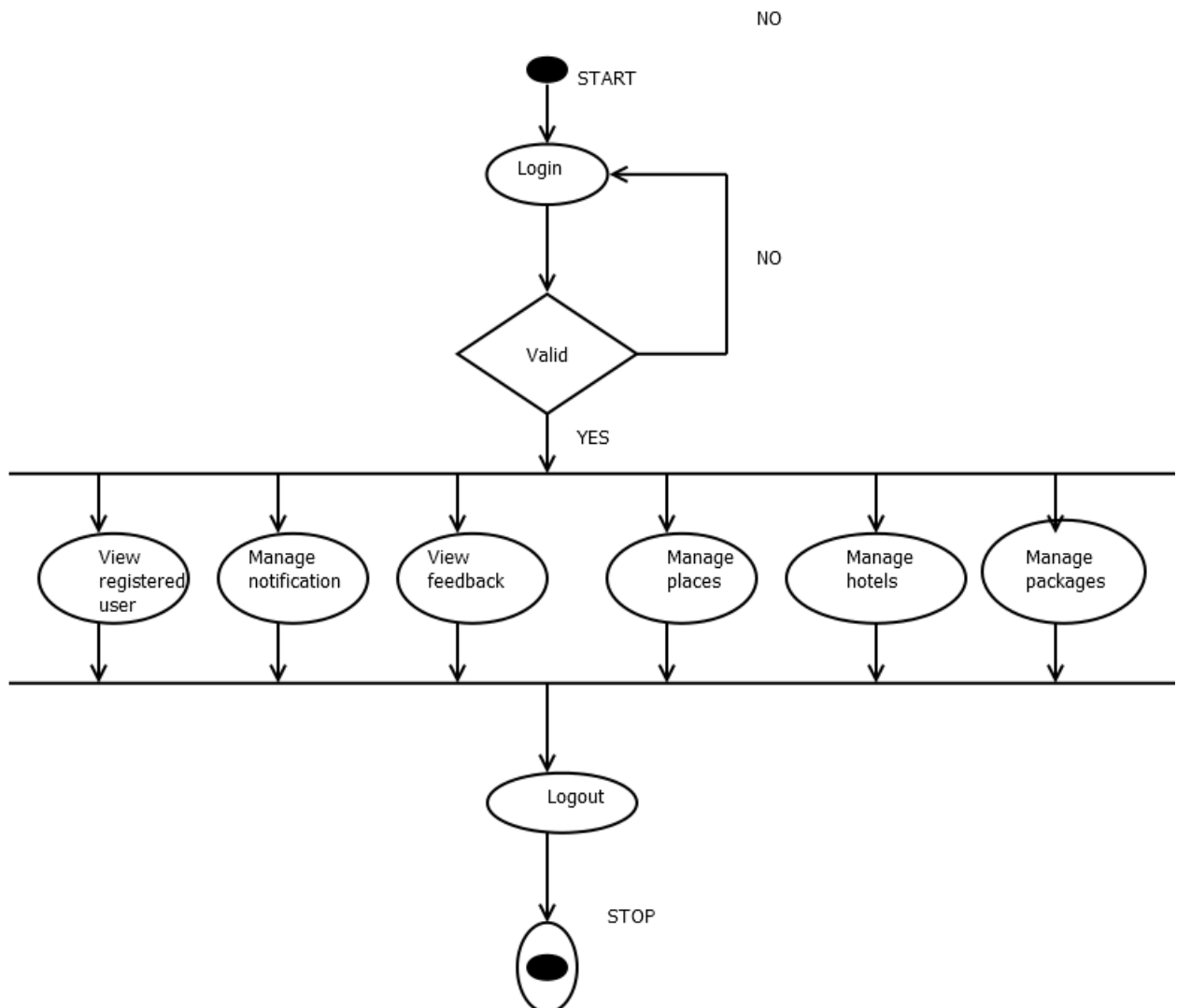


3.4 Object Oriented Design - UML Diagrams

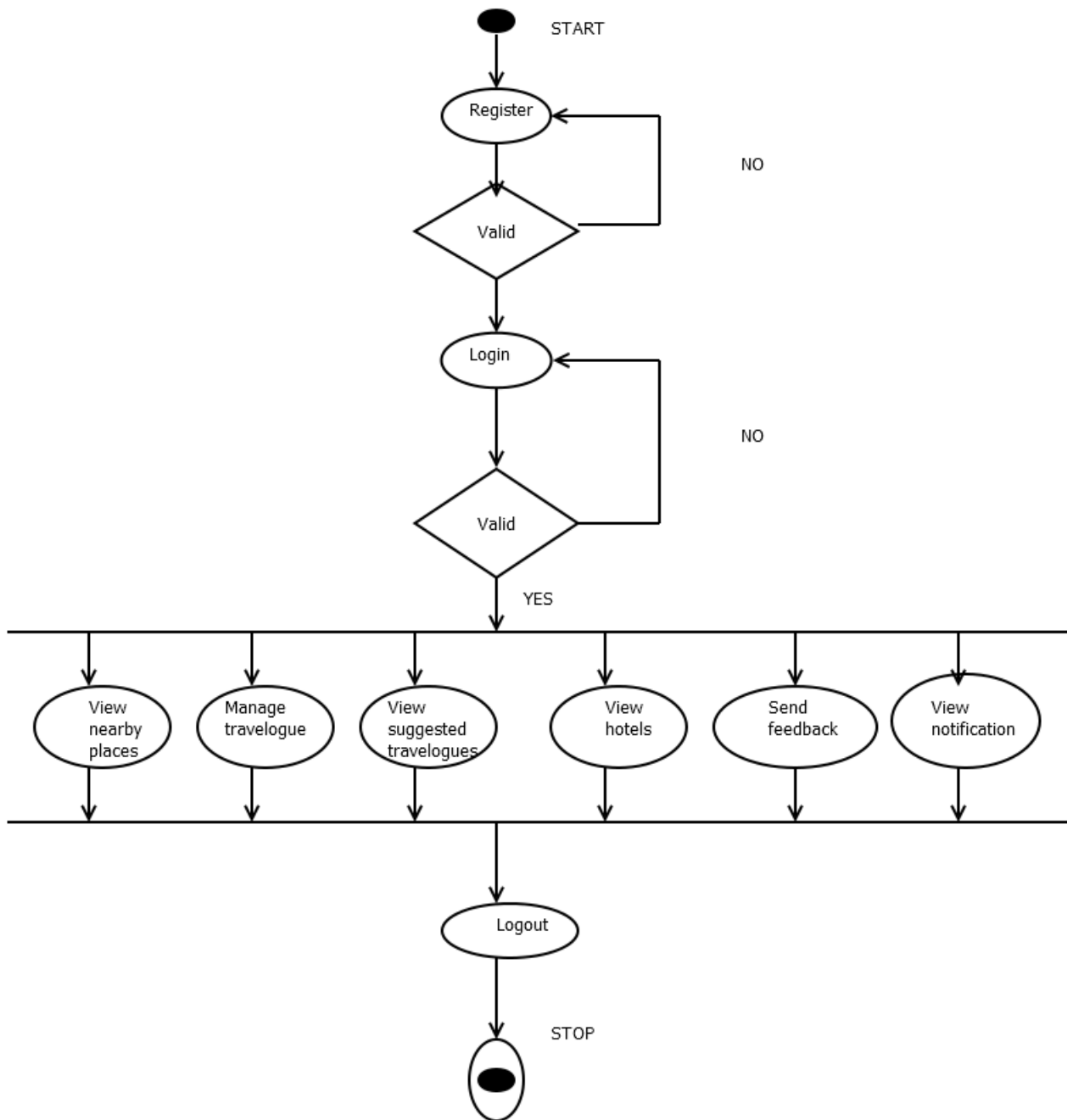
- **ACTIVITY DIAGRAM**

An activity diagram is a type of behavioral diagram in UML (Unified Modeling Language) used to visually represent the flow of control or the flow of activities within a system, process, or workflow. It's particularly useful for modeling the dynamic aspects of a system, showing how activities are coordinated and controlled as they progress from one to another.

ADMIN

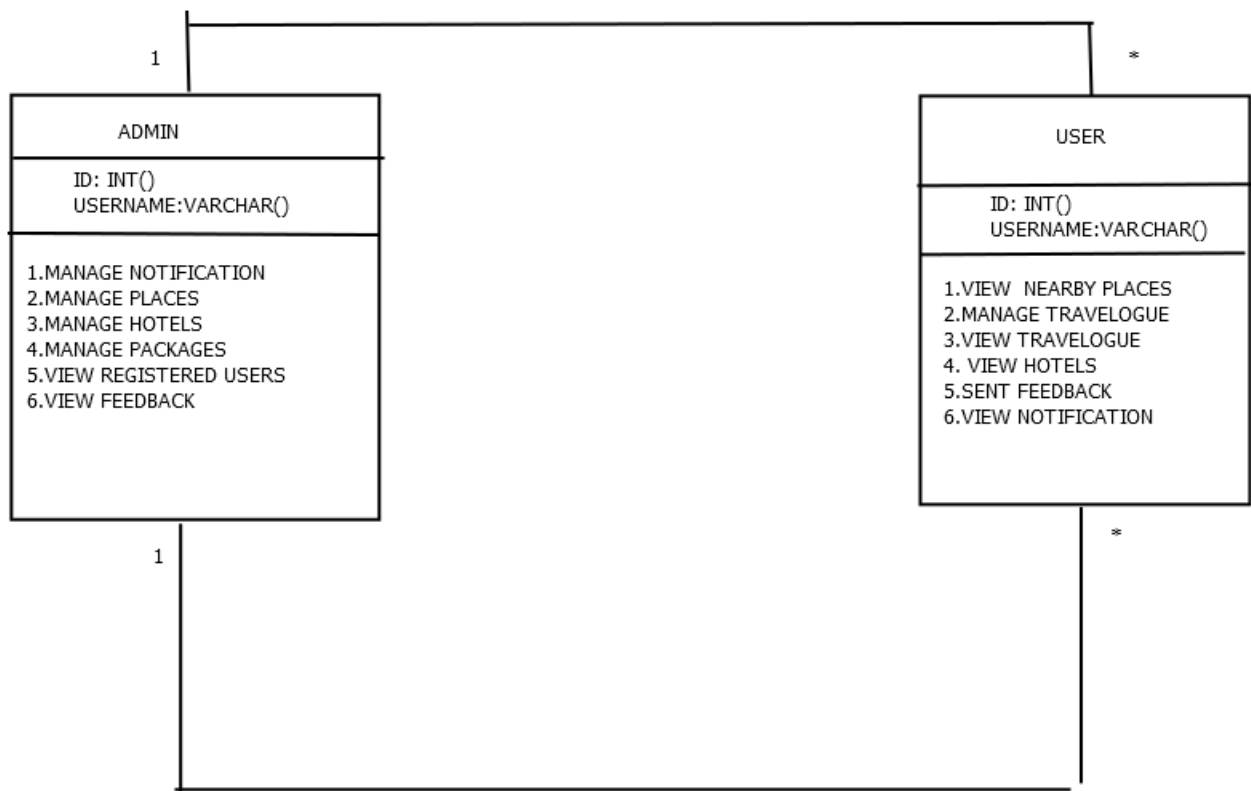


USER



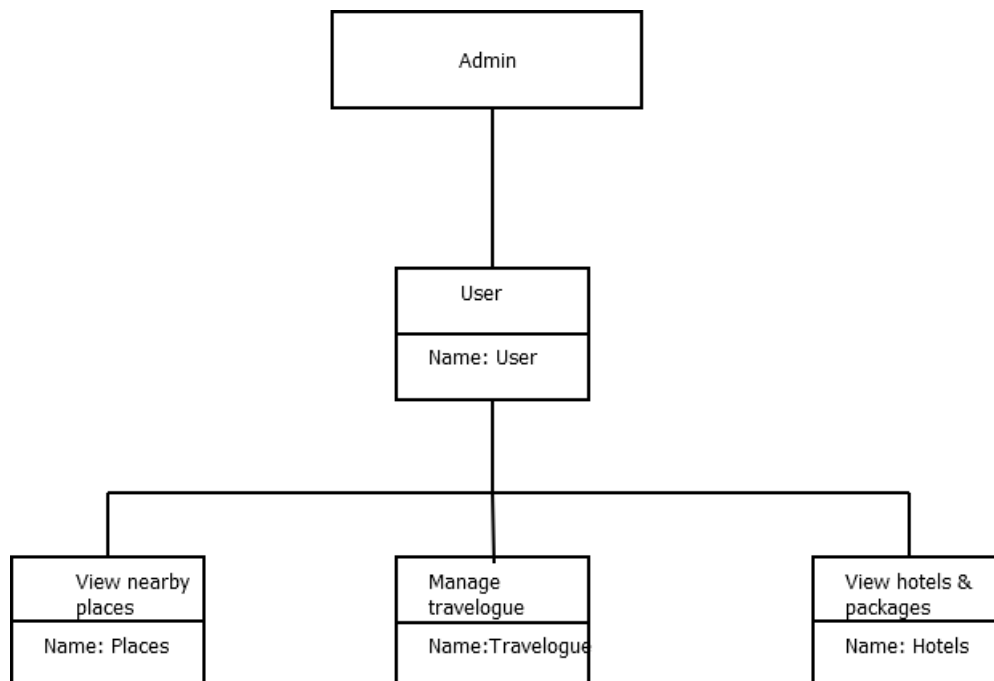
- **CLASS DIAGRAM**

A class diagram illustrates the structure and relationships between classes in an object-oriented system, showing attributes, methods, and associations between classes. It provides a blueprint for designing software systems, aiding in understanding the system's architecture and facilitating communication among developers.



- **OBJECT DIAGRAM**

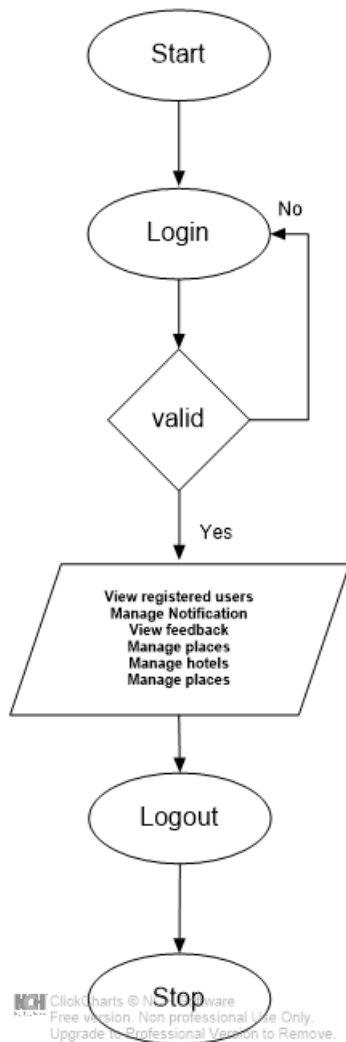
An object diagram in UML (Unified Modeling Language) is a structural diagram that shows a snapshot of the objects and their relationships at a particular point in time within a system. Unlike class diagrams, which represent classes and their relationships, object diagrams depict instances of classes (objects) and how they interact with each other.



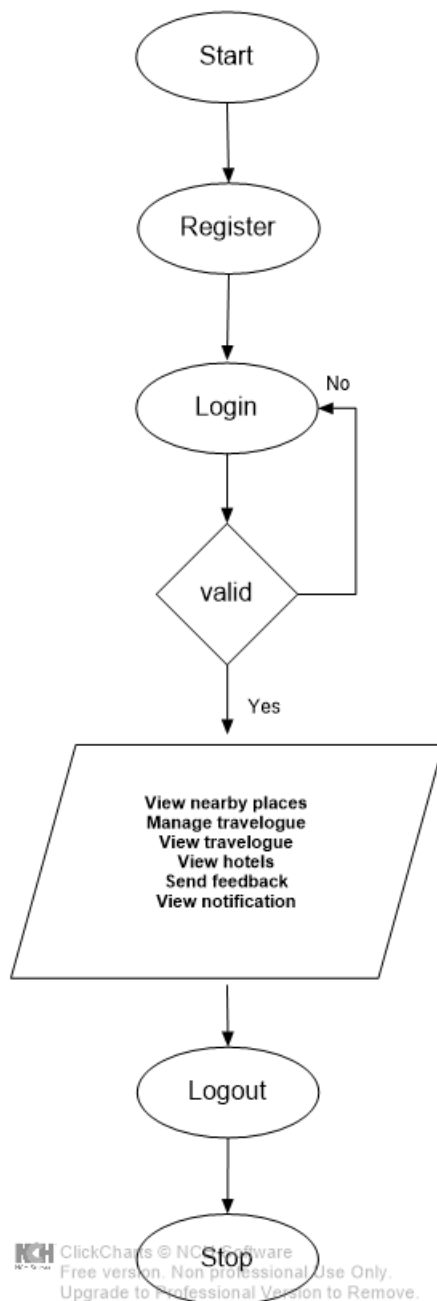
- **INFORMATION FLOW DIAGRAM**

An information flow diagram is a visual representation that illustrates the movement of data or information within a system or organization. It typically shows the flow of information between different components, processes, or entities involved in a particular system.

ADMIN

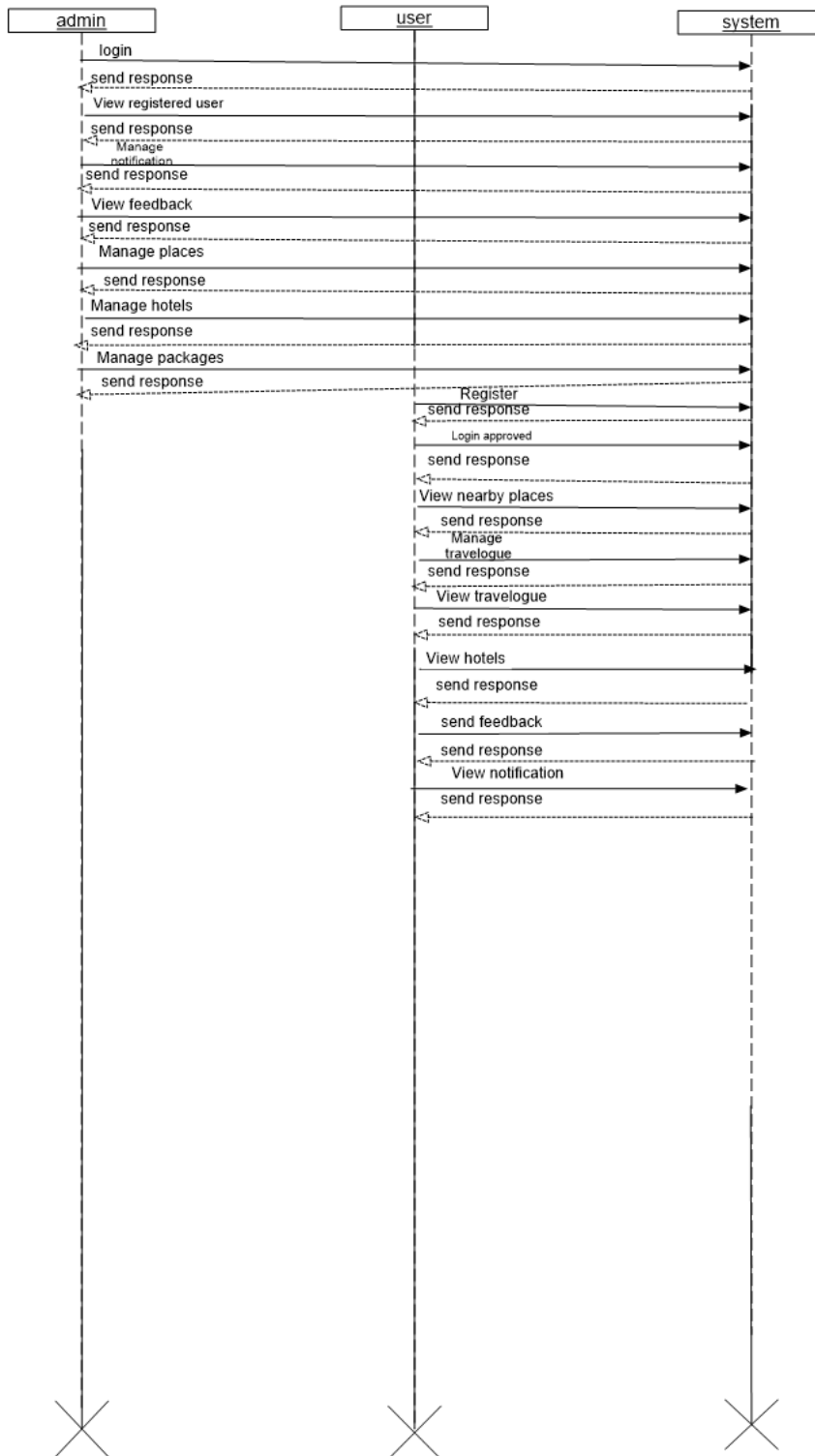


USER



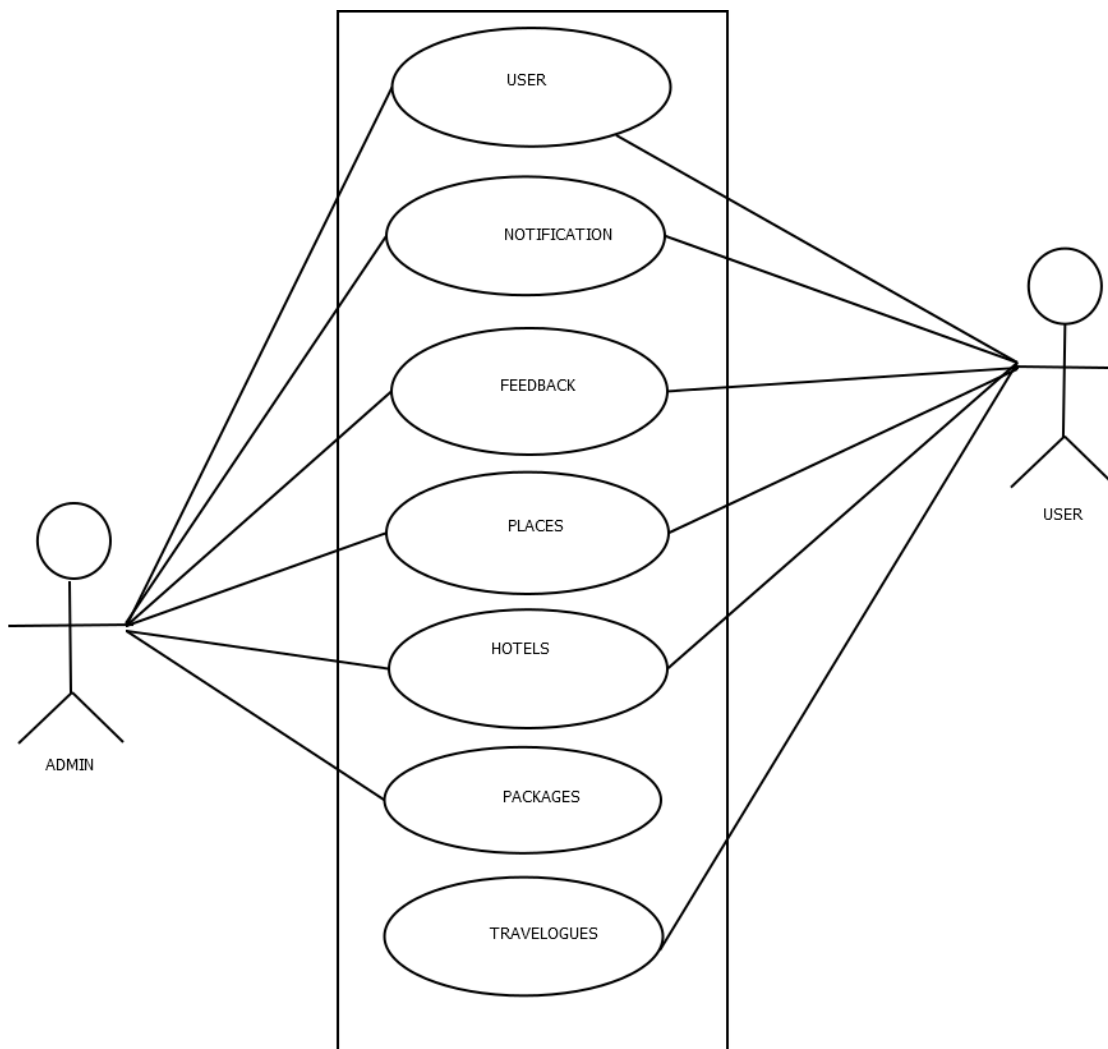
- **SEQUENCE DIAGRAM**

A sequence diagram is a type of interaction diagram that illustrates the flow of messages or interactions between objects or components within a system over time. It shows the chronological order of these interactions, indicating the sequence of actions that occur as different parts of the system communicate with each other to accomplish a specific task or scenario. Sequence diagrams are valuable for understanding the dynamic behavior of a system, identifying potential bottlenecks or communication issues, and aiding in the design and implementation of software systems.



- **USECASE DIAGRAM**

A use case diagram is a graphical representation that illustrates the interactions between users (actors) and a system. It depicts the various actions or functions, known as use cases, that users can perform within the system. These diagrams are valuable tools for capturing the requirements of a system and understanding the relationships between different actors and use cases. They provide a high-level view of the system's functionality and help stakeholders visualize how users interact with the system to achieve their goals.



3.5 MODULE DESCRIPTION

The three different modules included in the system are, admin, users and public. Different modules has their own function. The admin module controls the overall system.

ADMIN:

- User details registration
- Information about important places
- Sent notification (Relating new tour packages, new places etc.)
- View feedback
- Add hospitals
- Add hotels (With packages)

USERS:

- Add and view experience videos /description
- View nearby hotels (including different packages), hospitals etc.
- View nearby tourist attractions
- View and upload travelogue
- Suggest and view travelogue (Link to youtube)
- Sent feedback
- View notifications

PUBLIC:

- View important places
- View experience videos/ description
- View suggested view travelogue (Link to youtube)

CHAPTER 4

SYSTEM ENVIRONMENT

SYSTEM ENVIRONMENT

4.1 INTRODUCTION

System environment is the term commonly used to refer to support an application. A software environment for a particular application could include the operating system, specific development tools or compiler.

4.2 SOFTWARE REQUIREMENT SPECIFICATION

A software requirement specification (SRS), a requirements specification for a software system, is a complete description of the behavior of a system to be developed and may include a set of use cases that describe interactions the users will have with the software. In addition it also contains non-functional requirements. Non-functional requirements impose constraints on the design or implementation (such as performance engineering requirements, quality standards, or design constraints) the software requirements specification document enlists all necessary requirements that are required for the project development. To derive the requirements we need to have clear and thorough understanding of the products to be developed.

Operating System: WINDOWS 10 or above

Front end: HTML,CSS,JavaScript, Flutter (Mobile Application)

Back end: SQLyog

IDE: SubLimeText, WAMP, Android Studio

Language: Dart

Framework: Flutter

4.3 HARDWARE REQUIREMENTS SPECIFICATION

The most common set of requirements defined by any operating system or software

application is the physical computer resources, also known as hardware. A hardware requirements list is often accompanied by a hardware compatibility list (HCL), especially in case of operating systems. An HCL lists tested, compatible, and sometimes incompatible hardware devices for a particular operating system or application.

Processor: Intel Pentium or above.

Hard Disc: 320GB.

Display Type: PC Display.

Keyboard: PC/AT Enhanced PS/2Keyboard (110/10Key).

Mouse: First/Pilot Mouse Serial (c48).

Input Device: Mouse, keyboard

Output Device: Monitor, Mobile Display

4.3 TOOLS AND PLATFORMS

Flutter: Flutter is a growing mobile framework developed by Google, designed to help in the development of mobile and web applications with only one code base and the ability to export them as native apps for Android and iOS.

However, to develop an application supporting both the OSs, we need to code in two different languages using two different frameworks. To help overcome this complexity, there exists mobile frameworks supporting both OS. These frameworks range from simple HTML based hybrid mobile application framework (which uses HTML for User Interface and JavaScript for application logic) to complex language specific framework (which do the heavy lifting of converting code to native code). Irrespective of their simplicity or complexity, these frameworks always have many disadvantages, one of the main drawback being their slow performance.

Flutter also offers many ready to use widgets (UI) to create a modern application. These widgets are optimized for mobile environment and designing the application using widgets is as simple as designing HTML.

To be specific, Flutter application is itself a widget. Flutter widgets also supports animations and gestures. The application logic is based on reactive programming. Widget may optionally have a state. By changing the state of the widget, Flutter will automatically (reactive programming) compare the widget's state (old and new) and render the widget with only the necessary changes instead of re-rendering the whole widget.

SQLyog :SQLyog is a robust and user-friendly MySQL and MariaDB database management tool widely appreciated by database administrators and developers for its intuitive graphical interface and comprehensive feature set. It simplifies various aspects of database management, including design, development, and administration tasks, making it easier to manage large-scale databases efficiently. With SQLyog, users can perform complex queries, backup and restore databases, and synchronize data seamlessly. Additionally, it offers advanced functionalities such as query profiling, visual schema designing, and scheduled backups, enhancing productivity and ensuring data integrity. Its ability to connect to remote servers securely and support for a wide range of MySQL and MariaDB versions makes SQLyog a versatile tool for professionals seeking to optimize their database operations.

Windows 10: Windows 10 is a versatile and widely-used operating system developed by Microsoft, renowned for its user-friendly interface and robust performance. Introduced as a significant upgrade from its predecessors, Windows 10 integrates features designed to enhance both productivity and entertainment. It includes the Start Menu, which combines the traditional layout with live tiles for quick access to apps and information. The operating system supports a broad range of software applications and offers built-in security features such as Windows Defender Antivirus and BitLocker encryption to protect user data.

CHAPTER 5

SYSTEM IMPLEMENTATION

SYSTEM IMPLEMENTATION

5.1 INTRODUCTION

The system implementation phase of the project marks a pivotal stage where the meticulously planned designs and requirements are translated into a functional application. During this phase, developers harness the power of Flutter, an efficient and versatile framework, to craft a seamless and engaging user experience across multiple platforms. Leveraging Flutter's rich set of pre-designed widgets and its hot reload feature, developers iteratively build and refine the user interface, ensuring it meets the desired specifications and design guidelines. Beyond the user interface, the system implementation encompasses the integration of backend services, such as database management systems like MySQL, to store and manage product data, user information, and transactional records securely. Additionally, integration with APIs for payment processing, product catalog management, and order fulfillment further enriches the functionality of the online shopping platform.

5.2 CODING

▪ CODING STANDARDS:

Coding standards are a set of guidelines and best practices that programmers follow to write clean, consistent, and maintainable code and they lead to greater consistency. Consistency leads to code that is easier to understand, which in turn results in turn result in a code, which is easier to develop and maintain.

▪ SAMPLE CODES:

```
from flask import Flask
from public import public
from admin import admin
from api import api
from flask_cors import CORS
```

```
app=Flask(__name__)  
CORS(api)  
  
app.secret_key="tra"  
  
app.register_blueprint(public)  
app.register_blueprint(api,url_prefix='/api')  
app.register_blueprint(admin,url_prefix='/admin')  
  
app.run(debug=True,port="5524",host="0.0.0.0")
```

▪ **CODE VALIDATION AND OPTIMIZATION:**

Code validation and optimization are crucial steps in the software development lifecycle that ensure the reliability, efficiency, and performance of the final product. Validation involves checking the code for correctness, adherence to coding standards, and ensuring it meets the specified requirements. This process can include unit testing, integration testing, and code reviews, which help identify and rectify errors early in the development process.

5.3 DEBUGGING

Debugging techniques such as print statements, logging, and using debugging tools are essential for identifying and resolving issues in software development.

5.4 UNIT TESTING

Unit testing ensures that individual components of the system function correctly. A test plan is created outlining the testing approach, objectives, resources and schedule. Test cases are developed to validate each system functionality, covering scenarios such as user authentication, pdf upload, text, url and admin functionalities.

▪ **Test plan and test cases:**

Test Case Name: User Login Test

Description: Verify that users can successfully log in with valid credentials. Preconditions: User must be registered and logged out.

Test Steps: Navigate to the login page. Enter valid username and password. Click on the login button.

Expected Result: User should be redirected to the home page upon successful login.

CHAPTER 6

SYSTEM TESTING

SYSTEM TESTING

6.1 INTRODUCTION

System testing is a critical phase in the software development lifecycle where the complete and integrated software product is evaluated to ensure it meets the specified requirements. It includes integration testing to verify the interaction between components and system testing to validate the overall system behavior.

6.2 INTEGRATION TESTING

Data can be lost across an interface, one module can have an adverse effect on the other subfunctions, when combined they may not perform the desired functions. Integrated testing is the systematic testing to uncover the errors within the interface. This testing is done with simple data and the developed system has run successfully with this simple data. The need for integrated system is to the overall system performance.

The Modules of this project are connected and tested. After splitting the programs into units, the units were tested together to see the defects between each module and function. It is testing to one or more modules or functions together with the intent of interface defects between the modules or functions. Integration testing can involve putting together of groups of modules and functions with the goal of completing and verifying meets the system requirements.

6.3 SYSTEM TESTING

System testing focuses on testing the system as a whole to ensure that it meets the specified requirements and functions correctly in its intended environment. System Testing is a crucial step in Quality Management Process. In the Software Development

Life Cycle, System Testing is the first level where the System is tested as a whole.

The System is tested to verify whether it meets the functional and technical requirements.

- **Test plan and test cases**

Test case name: Viewing and Adding Travelogues

Description: A user wants to view travel experiences shared by others and add their own travelogue.

Preconditions: Ensure the user has registered and verified their account.

Test Steps:

Select a travelogue to view/listen/read.

Choose the option to add a new travelogue.

Submit the travelogue.

Expected Result: Users should be able to easily view existing travelogues and add their own, with the new content appearing correctly in the system.

CHAPTER 7

SYSTEM MAINTENANCE

SYSTEM MAINTENANCE

7.1 INTRODUCTION

Software Maintenance is the process of modifying a software product after it has been delivered to the client. The main purpose of software maintenance is to modify and update software application after delivery to correct faults and to improve performance.

7.2 MAINTENANCE

Software Maintenance must be performed in order to:

- Correct faults.
- Improve the design.
- Implement enhancements.
- Interface with other systems.
- Accommodate programs so that different hardware, software, system features, and telecommunications facilities can be used.
- Migrate legacy software.
- Retire software.

Categories of Software Maintenance: Maintenance can be divided into the following;

Corrective maintenance: Corrective maintenance of a software product may be essential either to rectify some bugs observed while the system is in use, or to enhance the performance of the system.

Adaptive maintenance: This includes modifications and updating when the customers need the product to run on new platforms, on new operating systems, or when they need the product to interface with new hardware and software.

Perfective maintenance: A software product needs maintenance to support the new features that the users want or to change different types of functionalities of the system according to the customer demands.

Preventive maintenance: This type of maintenance includes modifications and updates to prevent future problems of the software. It goals to attend problems, which are not significant at this moment but may cause serious issues in future.

Reverse Engineering – Reverse Engineering is processes of extracting knowledge or design information from anything man-made and reproducing it based on extracted information. It is also called back Engineering.

Software Reverse Engineering: Software Reverse Engineering is the process of recovering the design and requirements specification of a product from an analysis of its code. It is becoming important, since several existing software products, lack proper documentation, are highly unstructured, or their structure has degraded through a series of maintenance efforts.

CHAPTER 8

SYSTEM SECURITY MEASURES

SYSTEM SECURITY MEASURES

8.1 INTRODUCTION

Security applied to computing devices such as computers and smartphones, as well as computer networks such as private and public networks, including the whole Internet. The field covers all the processes and mechanisms by which digital equipment, information and services are protected from unintended or unauthorized access, change or destruction, and is of growing importance in line with the increasing reliance on computer systems of most societies worldwide. Computer security includes measures taken to ensure the integrity of files stored on computer or server as well as measures taken to prevent unauthorized access to stored data, by securing the physical perimeter of the computer equipment, authentication of users or computer accounts accessing the data, and providing a transmission

8.2 OPERATING SYSTEM LEVEL SECURITY

Operating system security (OS security) is the process of ensuring OS integrity, confidentiality and availability. OS security refers to specified steps or measures used to protect the OS from threats, viruses, worms, malware or remote hacker intrusions. OS security encompasses all preventive-control techniques, which safeguard any computer assets capable of being stolen, edited or deleted if OS security is compromised. OS security encompasses many different techniques and methods which ensure safety from threats and attacks. OS security allows different applications and programs to perform required tasks and stop unauthorized interference. OS security may be approached in many ways, including adherence to the following:

- Performing regular OS patch updates
- Installing updated antivirus engines and software
- Scrutinizing all incoming and outgoing network traffic through firewall

8.3 DATABASE LEVEL SECURITY

Authentication: Ensuring that only authorized users can access the database. This involves verifying the identity of users through credentials such as usernames and passwords.

Data Encryption: Encrypting sensitive data stored in the database to prevent unauthorized access in case of a breach. This can include encrypting data at rest and in transit using encryption algorithms like AES (Advanced Encryption Standard).

Database Access Policies: Implementing policies to limit direct access to the database server and instead require access through controlled interfaces such as APIs or application layers.

8.4 SYSTEM LEVEL SECURITY

System-level security refers to the architecture, policy and processes that ensure data and system security on individual computer systems. It facilitates the security of standalone and/or network computer systems/servers from events and processes that can exploit or violate its security or stature.

System-level security is part of a multi-layered security approach in which information security (IS) is implemented on an IT infrastructure's different components, layers or levels. System level security is typically implemented on end-user computer and server nodes. It ensures that system access is granted only to legitimate and trusted individuals and applications. The key objective behind system-level security is to keep system secure, regardless of security policies and processes at other levels. If other layers or levels are breached, the system must have the ability to protect itself.

CHAPTER 9
SYSTEM PLANNING AND SCHEDULING

SYSTEM PLANNING AND SCHEDULING

9.1 INTRODUCTION

System planning involves defining project goals, identifying requirements, and establishing a roadmap to guide project execution. It lays the foundation for the entire project lifecycle, providing a clear direction and strategy for project stakeholders to follow. Through meticulous planning, project teams can anticipate potential challenges, allocate resources effectively, and mitigate risks, thus setting the stage for project success.

9.2 PLANNING A SOFTWARE PROJECT

- Define Project Scope: Clearly define the objectives, features and functionalities and goals of the project
- Identify Requirements: Gather requirements from stakeholders, including user requirements, system functionalities and technical specifications.
- Resource Allocation: Allocate resources such as human resources, hardware and software tools required for system development.
- Create Work Breakdown Structure (WBS): Break down the project tasks into smaller, manageable components to facilitate planning and scheduling.
- Estimate Time and Effort: Estimate the time and effort required for each task or component based on historical data, expert judgment and past experiences.

9.3 STEPS INVOLVED IN PLANNING A SYSTEM

- Requirement Analysis: Gather and analyze user requirements, system functionalities and technical specifications.
- System Design: Design the system architecture, user interfaces and database schema based on the requirements.

- **Implementation:** Develop the system components, including frontend interfaces, backend logic and database functionality.
- **Testing:** Conduct unit testing, integration testing and system testing to validate the system functionality and performance.
- **Deployment:** Deploy the system in the production environment, ensuring proper configuration and setup.
- **Maintenance:** Establish procedures for ongoing maintenance, including software updates, bug fixes, and enhancements.
- **Evaluation and Feedback:** Continuously monitor the system's performance and collect feedback from users.

CHAPTER 10

SYSTEM COST ESTIMATION

SYSTEM COST ESTIMATION

10.1 INTRODUCTION

Cost estimation plays a pivotal role in the project lifecycle, serving as a foundation for budget planning, risk management, and project control. By accurately estimating costs upfront, organizations can avoid budget overruns, ensure resource availability, and improve project profitability. Additionally, cost estimation facilitates cost-benefit analysis, enabling stakeholders to evaluate the economic viability of proposed projects and make strategic investment decisions.

10.2 LOC BASED ESTIMATION

LOC (Lines of Code) based estimation is a widely used technique in software development for estimating project effort, duration, and cost based on the size of the codebase. It involves predicting the resources required to develop a software system by quantifying the number of lines of code that need to be written or modified.

The process of LOC-based estimation typically involves the following steps:

- ❖ Requirement Analysis: Gather and analyze project requirements to understand the scope and functionality of the software system.
- ❖ Breakdown of Requirements: Decompose high-level requirements into smaller, more manageable components such as modules, functions, or features.
- ❖ Estimation of LOC: Estimate the number of lines of code required to implement each component based on factors such as complexity, functionality, and programming language.

- ❖ Calculation of Effort: Convert the estimated LOC into effort hours or person-months using historical data, productivity metrics, and estimation models.
- ❖ Resource Allocation: Allocate resources such as developers, testers, and other team members based on the estimated effort and project schedule.
- ❖ Risk Assessment: Identify and assess potential risks and uncertainties that may impact the development process, such as technology constraints, dependencies, or changes in requirements.
- ❖ Validation and Adjustment: Validate the initial estimates against known benchmarks or industry standards and adjust as necessary based on feedback, changes in scope, or new information.

CHAPTER 11

FUTURE SCOPE AND FURTHER ENHANCEMENT

FUTURE SCOPE AND FURTHER ENHANCEMENT

11.1 INTRODUCTION

As urbanization and global mobility continue to rise, the need for reliable and efficient travel solutions becomes increasingly critical. Our proposed system addresses the fundamental challenges travelers face, such as navigating unfamiliar cities, missing transportation connections, and finding essential services. By providing comprehensive tracking capabilities, detailed location-based information, and user-generated travel experiences, our system acts as a vital guide for ensuring safe and enjoyable journeys.

While the current system already offers significant value through its user-friendly design and essential features, there is immense potential for further enhancement. To keep pace with evolving technologies and user expectations, it is crucial to explore additional functionalities and improvements. Future developments will focus on integrating advanced technologies, enhancing user personalization, and expanding the system's scope to offer an even more robust and versatile travel companion.

11.1 MERITS OF THE SYSTEM

➤ Enhanced Safety:

Real-Time Tracking: Provides real-time tracking capabilities that ensure travelers can monitor their routes and avoid getting lost or stranded.

Emergency Assistance: Offers quick access to emergency services and important contact numbers, enhancing safety during unforeseen situations.

➤ User-Friendly Navigation:

Detailed Maps and Directions: Delivers accurate maps and step-by-step directions, helping users navigate unfamiliar cities and find their way without hassle.

Bus and Station Alerts: Alerts users when they are approaching their bus stop or station,

reducing the likelihood of missing stops or getting off at the wrong location.

➤ Comprehensive Information:

Nearby Services: Lists details about nearby hotels, restaurants, hospitals, and other essential services, making it easier for users to find what they need quickly.

Tourist Attractions: Highlights nearby tourist attractions, allowing users to explore points of interest and make the most of their travels.

➤ Personalized Travel Experiences:

User-Generated Content: Features travelogues, videos, and audio recordings from other travelers, offering personal insights and recommendations.

Customizable Itineraries: Enables users to create and customize their itineraries based on personal preferences and interests.

➤ Cost-Effective Solution:

Low Cost: Designed to be affordable, ensuring that a wide range of users can benefit from its features without financial strain.

Easy Maintenance: Built with easy maintenance in mind, reducing the need for frequent updates or technical support.

➤ Interactive and Engaging:

Experience Sharing: Allows users to share their travel experiences, making the system more engaging and creating a sense of community among travelers.

Multimedia Integration: Supports videos, audios, and written travelogues, providing diverse ways for users to share and consume content.

➤ Accessibility:

Multilingual Support: Offers information in multiple languages, making it accessible to a global audience.

User-Friendly Interface: Designed with a simple and intuitive interface, ensuring that users of all ages and technical abilities can navigate the system with ease.

➤ **Informed Decision-Making:**

Tour Packages: Provides information on various tour packages available in nearby hotels, helping users choose the best options for their needs.

Reviews and Ratings: Includes reviews and ratings from other users, assisting in making informed decisions about services and attractions.

➤ **Community Building:**

Traveler Network: Facilitates connections between travelers, fostering a community where users can share tips, experiences, and support.

By incorporating these features, the proposed system not only addresses the practical challenges faced by travelers but also enriches their overall travel experience. The combination of safety, convenience, and community engagement makes it a valuable tool for anyone embarking on a journey, whether for leisure or business.

11.2 LIMITATIONS OF THE SYSTEM

➤ **Dependence on Internet Connectivity:**

Network Reliability: The system's real-time tracking, maps, and information services rely heavily on a stable internet connection. In areas with poor or no connectivity, the system's functionality may be significantly impaired.

Offline Access: Limited or no offline capabilities can hinder the usability of the system when users are in remote locations or face connectivity issues.

➤ **Data Accuracy and Timeliness:**

Outdated Information: The accuracy of information regarding hotels, restaurants, transport schedules, and tourist attractions depends on regular updates. Infrequent updates can lead to outdated or incorrect information being provided to users.

User-Generated Content Quality: The reliability of travelogues, reviews, and recommendations depends on the users. Inaccurate or biased content can affect the system's credibility.

➤ Language and Localization:

Language Barriers: Although the system supports multiple languages, it may not cover all languages or dialects, limiting accessibility for some users.

Cultural Relevance: Information and recommendations may not always be culturally relevant or appropriate for users from diverse backgrounds.

➤ Privacy and Security Concerns:

Data Privacy: The system requires access to personal data and location information, raising concerns about data privacy and security. Users may be hesitant to share their information due to potential misuse.

Security Risks: As with any online platform, the system is vulnerable to cyber threats, including hacking and data breaches, which can compromise user information.

➤ User Interface and Usability:

Complexity for Non-Tech-Savvy Users: Users who are not tech-savvy or are unfamiliar with modern apps may find the system complex and challenging to navigate.

Accessibility Issues: Users with disabilities may face difficulties if the system is not designed with accessibility features like screen readers, voice commands, and easy navigation.

➤ Cost and Resource Constraints:

Initial Setup and Maintenance: Although the system aims to be low-cost, the initial setup, regular maintenance, and updates can incur significant costs and require resources that may not be readily available.

Hardware Requirements: Users need compatible devices with sufficient storage and processing power to effectively run the application, which may not be accessible to everyone.

➤ Integration with Local Services:

Limited Integration: Integration with local public transport systems, emergency

services, and other local amenities may be limited or unavailable in certain regions, reducing the system's effectiveness.

Regulatory and Compliance Issues: Adhering to local regulations and compliance standards can be challenging, especially in regions with stringent data protection laws.

➤ **User Engagement and Content Quality:**

Engagement Levels: Sustaining high levels of user engagement and encouraging users to contribute high-quality content can be challenging.

Moderation and Verification: Ensuring the authenticity and accuracy of user-generated content requires robust moderation and verification processes, which can be resource-intensive.

11.3 FUTURE ENHANCEMENT OF THE SYSTEM

The proposed travel assistance system already offers numerous valuable features to ensure safe and enjoyable journeys for travelers. However, to keep pace with evolving technologies and user expectations, several future enhancements can be considered. These enhancements aim to further improve the system's functionality, user experience, and overall utility.

➤ **Integration with Public Transport Systems:**

Real-Time Public Transport Tracking: Integrate with local public transport systems to provide real-time tracking of buses, trains, and other forms of public transportation. This can help users avoid missing their buses and accurately track their journeys.

Ticket Booking and Payment Integration: Allow users to book and pay for tickets within the app, making the travel experience more seamless.

➤ **Advanced Navigation Features:**

Augmented Reality (AR) Navigation: Implement AR to enhance the navigation

experience, especially in crowded or complex areas like airports and bus stations.

Offline Maps and Guides: Enable offline access to maps and travel guides, ensuring users can navigate even without an internet connection.

➤ **Enhanced Personalization:**

AI-Based Recommendations: Utilize AI to offer personalized recommendations based on user preferences, past behaviors, and current trends. This can include customized travel itineraries, dining suggestions, and nearby attractions.

User Profiles and Preferences: Allow users to set up profiles with their travel preferences, dietary restrictions, and interests to tailor the recommendations further.

➤ **Safety and Emergency Features:**

Emergency Assistance: Add features for emergency assistance, including quick-dial options for local emergency services, embassies, and consulates.

Safety Alerts and Notifications: Provide real-time safety alerts about the current location, including weather warnings, political unrest, or public health advisories.

➤ **Social and Community Features:**

Social Sharing and Networking: Integrate social networking features to allow users to share their experiences, connect with fellow travelers, and join travel communities.

Reviews and Ratings: Enable users to leave reviews and ratings for hotels, restaurants, attractions, and transport services, helping build a community-driven resource base.

➤ **Multilingual Support:**

Language Translation: Incorporate language translation features for both text and speech to assist users in communicating effectively in foreign countries.

Localized Content: Provide localized content in multiple languages to cater to a global audience.

CHAPTER 12

CONCLUSION

CONCLUSION

In conclusion, the proposed system addresses the critical issue of ensuring a safe and informed journey for millions of people traveling to unfamiliar cities. By providing comprehensive tracking capabilities, the system helps users avoid common problems such as missing buses, boarding the wrong bus, or disembarking at the wrong station. The integration of detailed information about places, important locations, maps, nearby hotels, restaurants, available tour packages, hospitals, and other tourist attractions makes it an invaluable tool for travelers.

Moreover, the inclusion of user-generated content such as experience videos, audios, and travelogues adds a unique and engaging dimension, offering firsthand insights and recommendations. This feature not only enhances the overall travel experience but also serves as a practical guide for new travelers. The system's low cost and easy maintenance further ensure its accessibility and sustainability.

Overall, this innovative tourist guide system promises to make travel safer, more enjoyable, and richly informative, catering to the diverse needs of travelers and ensuring they have a reliable companion throughout their journey.

CHAPTER 13

BIBILIOGRAPHY

BIBLIOGRAPHY

- Buhalis, D., & Amaranggana, A. (2014). Smart Tourism Destinations. In *Information and Communication Technologies in Tourism 2014* (pp. 553-564). Springer, Cham.
- Gretzel, U., Sigala, M., Xiang, Z., & Koo, C. (2015). Smart tourism: foundations and developments. *Electronic Markets*, 25(3), 179-188.
- Koo, C., Shin, S., Gretzel, U., & Hunter, W. C. (2016). Conceptualization of smart tourism destination competitiveness. *Asia Pacific Journal of Information Systems*, 26(4), 561-576. (<https://tourism.io/travel>)
- Neuhofer, B., Buhalis, D., & Ladkin, A. (2015). Smart technologies for personalized experiences: A case study in the hospitality domain. *Electronic Markets*, 25(3), 243-254.
- Yoo, K. H., & Gretzel, U. (2016). Use and creation of social media by travelers. In *Managing Customer Experiences in an Omnichannel World: Melody of Online and Offline Environments* (pp. 409-431). Springer, Cham.

CHAPTER 14
APPENDIX

Main.py

```
from flask import Flask
from public import public
from admin import admin
from api import api
from flask_cors import CORS

app=Flask(__name__)
CORS(api)

app.secret_key="tra"

app.register_blueprint(public)
app.register_blueprint(api,url_prefix='/api')
app.register_blueprint(admin,url_prefix='/admin')

app.run(debug=True,port="5524",host="0.0.0.0")
```

Admin.py

```
from flask import *

from database import *

import uuid

admin=Blueprint('admin',__name__)

@admin.route('admin_home',methods=['get','post'])
def admin_home():
```

```

if not session.get("login_id") is None:
    return render_template("admin_home.html")
else:
    return redirect(url_for("public.login"))

@admin.route('admin_view_reg_user',methods=['get','post'])
def admin_view_reg_user():
    data={}
    if not session.get("login_id") is None:
        q="SELECT *,concat(first_name,' ',last_name) as user_name FROM `users`"
        res=select(q)
        data['view_user']=res

    return render_template('admin_view_reg_user.html',data=data)
else:
    return redirect(url_for("public.login"))

@admin.route('admin_manage_place_type',methods=['get','post'])
def admin_manage_place_type():
    if not session.get("login_id") is None:
        data={}
        if 'action' in request.args:
            action=request.args['action']
            tid=request.args['tid']
        else:
            action=None
        print(action)

        if action=="delete":
            q="DELETE FROM `place_type` WHERE `place_type_id`='%s'"%(tid)
            delete(q)
            return redirect(url_for('admin.admin_manage_place_type'))

        if 'type' in request.form:
            p_type=request.form['p_type']

```

```

        q="INSERT INTO `place_type` VALUES(NULL,'%s')"%(p_type)
        insert(q)

    q="SELECT * FROM `place_type`"
    res=select(q)
    data['view_type']=res

    return render_template('admin_manage_place_type.html',data=data)
else:
    return redirect(url_for("public.login"))

@admin.route('admin_manage_place',methods=['get','post'])
def admin_manage_place():
    if not session.get("login_id") is None:
        data={}
        if 'action' in request.args:
            action=request.args['action']
            id=request.args['id']
        else:
            action=None
        print(action)

        if action=="delete":
            q="DELETE FROM `places` WHERE `place_id`='%s'"%(id)
            delete(q)
            return redirect(url_for('admin.admin_manage_place'))

        if action=="update":
            q="SELECT * FROM `places` INNER JOIN `place_type` ON
`place_type_id`=`type_id` WHERE `place_id`='%s'"%(id)
            res=select(q)
            data['uplace']=res

        if 'updplace' in request.form:

```

```

        title=request.form['title']
        des=request.form['des']
        lati=request.form['lati']
        longi=request.form['longi']
        q="UPDATE `places` SET
`latitude`='%s',`longitude`='%s',`title`='%s',`description`='%s' WHERE
`place_id`='%s'"%(lati,longi,title,des,id)
        update(q)
        return redirect(url_for('admin.admin_manage_place'))

    if 'place' in request.form:
        ptype=request.form['ptype']
        title=request.form['title']
        des=request.form['des']
        lati=request.form['lati']
        longi=request.form['longi']
        photo=request.files['photo']
        path='static/uploads/'+str(uuid.uuid4())+photo.filename
        photo.save(path)
        place=request.form['place']
        q="INSERT INTO `places`
VALUES(NULL,'%s','%s','%s','%s','%s','%s')"%(lati,longi,ptype,title,des,path)
        insert(q)

        q="SELECT * FROM `place_type`"
        res=select(q)
        data['p_type']=res

        q="SELECT * FROM `places` INNER JOIN `place_type` ON
`place_type_id`=`type_id`"
        res=select(q)
        data['view_plac']=res

        return render_template('admin_manage_place.html',data=data)
    else:
        return redirect(url_for("public.login"))

```

```

@admin.route('admin_view_feedback',methods=['get','post'])
def admin_view_feedback():
    if not session.get("login_id") is None:
        data={}
        q="SELECT *,CONCAT(`first_name`,`last_name`) AS user_name FROM
`feedback` INNER JOIN `users` USING(`login_id`)"
        res=select(q)
        data['view_feed']=res

        return render_template('admin_view_feedback.html',data=data)
    else:
        return redirect(url_for("public.login"))

```

```

@admin.route('admin_manage_hotels',methods=['get','post'])
def admin_manage_hotels():
    if not session.get("login_id") is None:

        data={}

        if 'action' in request.args:
            action=request.args['action']
            id=request.args['id']
        else:
            action=None
        print(action)

        if action=="delete":
            q="DELETE FROM `hotels` WHERE `hotel_id`='%s'"%(id)
            delete(q)
            return redirect(url_for('admin.admin_manage_hotels'))

        if action=="update":
            q="SELECT * FROM `hotels` WHERE `hotel_id`='%s'"%(id)

```



```

        res=select(q)
        data['uhotel']=res

    if 'uphotel' in request.form:
        hname=request.form['hname']
        about=request.form['about']
        lati=request.form['lati']
        longi=request.form['longi']
        phone=request.form['phone']
        email=request.form['email']
        place=request.form['place']

        q="UPDATE `hotels` SET
`hotel_name`='%s',`about`='%s',`latitude`='%s',`longitude`='%s',`phone`='%s',`email`='%s',
`place`='%s' WHERE `hotel_id`='%s'"%(hname,about,lati,longi,phone,email,place,id)
        update(q)
        return redirect(url_for('admin.admin_manage_hotels'))

    if 'hotel' in request.form:
        hname=request.form['hname']
        about=request.form['about']
        lati=request.form['lati']
        longi=request.form['longi']
        phone=request.form['phone']
        email=request.form['email']
        photo=request.files['photo']
        path='static/uploads/'+str(uuid.uuid4())+photo.filename
        photo.save(path)
        place=request.form['place']

        q="INSERT INTO `hotels`
VALUES(NULL,'%s','%s','%s','%s','%s','%s','%s','%s')"%(hname,about,lati,longi,phone,em
ail,path,place)
        insert(q)

    q="SELECT * FROM `hotels`"
    res=select(q)

```

```

        data['view_hotel']=res

    return render_template("admin_manage_hotels.html",data=data)
else:
    return redirect(url_for("public.login"))

@admin.route("admin_manage_packages",methods=['get','post'])
def admin_manage_packages():
    if not session.get("login_id") is None:
        data={}

        q="select * from hotels"
        res=select(q)
        data['view_hotels']=res

        q="select * from packages inner join hotels using(hotel_id)"
        res=select(q)
        data['view_pack']=res

        if 'pack' in request.form:
            hotel_id=request.form['hotel_id']
            title=request.form['title']
            des=request.form['des']
            amount=request.form['amount']

            q="INSERT INTO `packages`
VALUES(NULL,'%s','%s','%s','%s')"%(hotel_id,title,des,amount)
            insert(q)

            return redirect(url_for("admin.admin_manage_packages"))

        if 'action' in request.args:
            action=request.args['action']
            id=request.args['id']

```

```

else:
    action=None
print(action)

if action=="delete":
    q="delete from packages where package_id='%s'"%(id)
    delete(q)
    return redirect(url_for("admin.admin_manage_packages"))

if action=="update":
    q="select * from packages inner join hotels using(hotel_id) where
package_id='%s'"%(id)
    res=select(q)
    data['upack']=res

if 'upack' in request.form:
    title=request.form['title']
    des=request.form['des']
    amount=request.form['amount']

    q="UPDATE `packages` SET `title`='%s',`description`='%s',`amount`='%s'
WHERE `package_id`='%s'"%(title,des,amount,id)
    update(q)
    return redirect(url_for("admin.admin_manage_packages"))

return render_template("admin_manage_packages.html",data=data)
else:
    return redirect(url_for("public.login"))

@admin.route("admin_manage_ntification",methods=['get','post'])
def admin_manage_ntification():
    if not session.get("login_id") is None:
        data={}
        q="SELECT * FROM `users`"
        res=select(q)

```

```

data['vendors']=res

if 'noti' in request.form:
    title=request.form['title']
    des=request.form['des']

    q="INSERT INTO `notifications`
VALUES(NULL,'%s','%s',now())"%(title,des)
    insert(q)

if 'action' in request.args:
    action=request.args['action']
    id=request.args['id']
else:
    action=None
print(action)

if action=='update':
    w=" select * from notifications where notification_id='%s' "%(id)
    data['upd']=select(w)

if 'upd' in request.form:
    tit=request.form['titl']
    dcr=request.form['desc']
    ud="update `notifications` set title='%s' , description='%s' where
notification_id='%s' "%(tit,dcr,id)
    update(ud)
    flash('updated successfully...')
    return redirect(url_for("admin.admin_manage_ntification"))

if action=="delete":
    q="DELETE FROM `notifications` WHERE `notification_id`='%s' "%(id)
    delete(q)
    return redirect(url_for("admin.admin_manage_ntification"))

```

```

q="SELECT * FROM `notifications`"
res=select(q)
data['view_noti']=res

return render_template("admin_manage_ntification.html",data=data)
else:
    return redirect(url_for("public.login"))

@admin.route("adminviewtravalogues",methods=['get','post'])
def adminviewtravalogues():
    if not session.get("login_id") is None:
        data={}

        q="SELECT *,concat(first_name,' ',last_name)as name FROM `travelogue` inner
join login on travelogue.login_id=login.login_id inner join users on
login.login_id=users.login_id"
        res=select(q)
        data['viewtravalogue']=res
        if 'action' in request.args:
            action=request.args['action']
            id=request.args['travelogue_id']
        else:
            action=None
        print(action)

        if action=='active':
            w=" UPDATE `travelogue` SET `t_status`='public' WHERE
`travelogue_id`='%s'"%(id)
            update(w)
            return redirect(url_for("admin.adminviewtravalogues"))

```

```

        return render_template("adminviewtravalogues.html",data=data)
    else:
        return redirect(url_for("public.login"))

@admin.route("adminviewtravelogueimages",methods=['get','post'])
def adminviewtravelogueimages():
    if not session.get("login_id") is None:
        data={}
        trav_id=request.args['tid']

        if 'idds' in request.args:
            idds=request.args['idds']
            q="delete from uploads where upload_id='%s'" %(idds)
            delete(q)
            return redirect(url_for("admin.adminviewtravelogueimages",tid=trav_id))
        data['val']=trav_id
        q="SELECT * FROM `uploads` WHERE `travelogue_id`='%s'"%(trav_id)
        res=select(q)
        data['viewtravalogue']=res

    return render_template("adminviewtravelogueimages.html",data=data)
else:
    return redirect(url_for("public.login"))

```

Api.py

```

from flask import *
from database import *
import qrcode
import demjson
import uuid

```

```

api=Blueprint('api',__name__)


@api.route('/login', methods=['post'])
def login():
    username = request.form['uname']
    password = request.form['password']
    print(username)
    print(password)
    q="select * from login where username='%s' and
password='%s'"%(username,password)
    res=select(q)
    print(res)
    if res:
        return jsonify(status="success", lid=res[0]['login_id'], type=res[0]["usertype"])
    else:
        return jsonify(status="failure")


@api.route('/user_registration',methods=['post'])
def user_registration():
    data={}
    fname=request.form['fname']
    lname=request.form['lname']
    phone=request.form['phone']
    address=request.form['address']
    place=request.form['place']
    pin=request.form['pin']
    email=request.form['email']
    passw=request.form['passw']
    qr="SELECT * FROM `login` WHERE `username`='%s' OR
`password`='%s'"%(email,passw)
    res=select(qr)
    if res:
        data['status']='duplicate'

```

```

        return jsonify(status="duplicate")
    else:
        q="INSERT INTO `login` VALUES(NULL,'%s','%s','user')"%(email,passwd)
        val=insert(q)
        qr="INSERT INTO `users`
VALUES(NULL,'%s','%s','%s','%s','%s','%s','%s','%s','0','0')"%(val,fname,lname,address,pi
n,place,phone,email)
        insert(qr)
        return jsonify(status="success")

@api.route('/User_view_places', methods=['POST'])
def User_view_places():
    q = "SELECT * FROM `place_type` INNER JOIN `places` ON
`place_type`.`place_type_id`=`places`.`type_id`"
    res = select(q)
    print(q)
    return jsonify(status="ok", data=res)

@api.route('/useradd_to_fever', methods=['POST'])
def useradd_to_fever():
    data = {}
    lid = request.form['lid']
    place_id = request.form['place_id']

    # Check if the place is already a favorite for the user
    query = "SELECT * FROM `interests` WHERE `user_id`=(SELECT `user_id` FROM
`users` WHERE `login_id`='%s') AND `place_id`='%s'" % (lid, place_id)
    res = select(query)

    if res:
        # Place is already a favorite
        data['status'] = "color is permenet"
    else:
        # Place is not a favorite, add it to favorites
        insert_query = "INSERT INTO `interests` VALUES (NULL, (SELECT `user_id`
FROM `users` WHERE `login_id`='%s'), '%s')" % (lid, place_id)

```



```

        insert(insert_query)
        data['status'] = 'success'

    return jsonify(data)

@api.route('/userremove_from_fever', methods=['POST'])
def userremove_from_fever():
    data = {}
    lid = request.form['lid']
    place_id = request.form['place_id']
    query = "DELETE FROM `interests` WHERE `user_id` = (SELECT `user_id` FROM `users` WHERE `login_id`='%s') AND `place_id` = '%s'"%(lid,place_id)
    delete(query)
    data['status'] = 'success'
    return jsonify(data)

@api.route('/User_view_hotels', methods=['POST'])
def User_view_hotels():
    q = "SELECT * FROM `hotels`"
    res = select(q)
    print(q)
    return jsonify(status="ok", data=res)

@api.route('/User_view_hotelpackeges', methods=['POST'])
def User_view_hotelpackeges():
    hotel_id = request.form['hotel_id']
    q = "SELECT * FROM `hotels` INNER JOIN `packages` USING(`hotel_id`) WHERE `hotel_id`='%s'"%(hotel_id)

    res = select(q)
    print(q)
    return jsonify(status="ok", data=res)

```

```

@api.route('/usersendfeedback',methods=['post'])
def usersendfeedback():
    data={}
    login_id=request.form['lid']
    feed_des=request.form['feedback']
    title=request.form['description']
    q= "INSERT INTO `feedback` VALUES(NULL,(SELECT `user_id` FROM `users`
WHERE `login_id`='%s'), '%s', '%s', NOW())"% (login_id,title,feed_des)
    print(q)
    id=insert(q)
    return jsonify(status="success")

```

```

@api.route('/userviewfeedback',methods=['post'])
def userviewfeedback():
    data = {}

    log_id=request.form['lid']

    q="SELECT * FROM `feedback` WHERE `login_id`=(SELECT `user_id` FROM
`users` WHERE `login_id`='%s')"% (log_id)
    result = select(q)
    return jsonify(status="ok", data=result)

```

```

@api.route('/user_view_noti',methods=['post'])
def user_view_noti():
    data = {}
    log_id=request.form['lid']
    qr="SELECT * FROM `notifications` "
    result = select(qr)
    return jsonify(status="ok", data=result)

```

```

@api.route('/Customer_send_complaint',methods=['post'])
def Customer_send_complaint():

```

```

data={}
log_id=request.form['lid']
complaint=request.form['complaint']
q="INSERT INTO `complaints` VALUES(NULL,(SELECT `user_id` FROM `users`
WHERE `login_id`='%s'),'%s','pending',curdate()))"%(log_id,complaint)
print(q)
res=insert(q)
if res:
    return jsonify(status="success")
else:
    return jsonify(status="failed")

```

```

@api.route('/view_complaint',methods=['post'])
def view_complaint():
    data={}
    log_id=request.form['lid']
    q="SELECT * FROM `complaints` WHERE `user_id`=(SELECT `user_id` FROM
`users` WHERE `login_id`='%s')"%(log_id)
    res=select(q)
    if res:
        return jsonify(status="ok", data=res)
    else:
        return jsonify(status="nodata")

```

```

@api.route('/user_add_travelogue',methods=['post'])
def user_add_travelogue():
    data={}
    loginid=request.form['lid']

```

```

    p_name=request.form['place']
    title=request.form['title']
    des=request.form['description']
    q= "INSERT INTO `travelogue`
VALUES(NULL,'%s','%s','%s','%s',NOW(),'block')"%(loginid,p_name,title,des)
    print(q)
    id=insert(q)
    return jsonify(status="success")

```

```

@api.route('/user_view_travelogue',methods=['post'])
def user_view_travelogue():
    data = {}
    loginid=request.form['lid']
    qr="SELECT * FROM `travelogue` WHERE `login_id`='%s' and
`t_status`='public'"%(loginid)
    result = select(qr)
    return jsonify(status="ok", data=result)

```

```

@api.route('/deletetravelogue', methods=['POST'])
def deletetravelogue():
    data = {}
    travelogue_id = request.form['travelogue_id']
    query = "DELETE FROM `travelogue` WHERE `travelogue_id`='%s'"%(travelogue_id)
    delete(query)
    qr="DELETE FROM `uploads` WHERE `travelogue_id`='%s'"%(travelogue_id)
    delete(qr)
    data['status'] = 'success'
    return jsonify(data)

```

```

@api.route('/user_upload_file',methods=['post'])
def user_upload_file():

```

```

    data={}

```

```

path = ""
trv_id=request.form['travelogue_id']
yts=request.form['yts']
desc=request.form['desc']
logid=request.form['lid']
image=request.files['file']
ftype = request.form['ftype']

if ftype == 'video':
    path='static/blogs/'+str(uuid.uuid4())+ ".mp4"
    image.save(path)
else:
    path='static/blogs/'+str(uuid.uuid4())+ ".jpg"
    image.save(path)
q="INSERT INTO `uploads`
VALUES(NULL,'%s','%s','%s','%s','%s','%s')"%(trv_id,logid,path,yts,desc,ftype)
print(q)
res=insert(q)
if res:
    return jsonify(status="success")
else:
    return jsonify(status="failure")

@api.route('/myuser_view_videos', methods=['post'])
def myuser_view_videos():
    data = {}
    trav_id = request.form['travelogueId']
    # qr = "SELECT * FROM `uploads` WHERE `travelogue_id`='%s'" % (trav_id)
    qr = "SELECT * FROM `uploads` where upload_type='video' and
`travelogue_id`='%s'"%(trav_id)
    print(qr)
    result = select(qr)
    return jsonify(status="ok", data=result)

@api.route('/myuser_delete_videos', methods=['post'])
def myuser_delete_videos():

```

```

data={}
upload_id=request.form['upload_id']
qr="DELETE FROM `uploads` WHERE `upload_id`='%s'"%(upload_id)
delete(qr)
return jsonify(status="ok")

```

```

@api.route('/myUser_view_travel_images', methods=['post'])
def myUser_view_travel_images():
    data = {}
    trav_id = request.form['travelogueId']
    # qr = "SELECT * FROM `uploads` WHERE `travelogue_id`='%s'" % (trav_id)
    qr = "SELECT * FROM `uploads` where upload_type='image' and
`travelogue_id`='%s'"%(trav_id)
    print(qr)
    result = select(qr)
    return jsonify(status="ok", data=result)

```

```

@api.route('/myuser_delete_images', methods=['post'])
def myuser_delete_images():
    data={}
    upload_id=request.form['upload_id']
    qr="DELETE FROM `uploads` WHERE `upload_id`='%s'"%(upload_id)
    delete(qr)
    return jsonify(status="ok")

```

```

@api.route('/My_view_travelogue',methods=['post'])
def My_view_travelogue():
    data = {}
    loginid=request.form['lid']
    qr="SELECT * FROM `travelogue` WHERE `login_id`='%s' and
`t_status`='public'"%(loginid)
    result = select(qr)

```

```
return jsonify(status="ok", data=result)
```

```
@api.route('/user_view_images_videos', methods=['post'])
def user_view_images_videos():
    data = {}
    trav_id = request.form['travelogueId']
    # qr = "SELECT * FROM `uploads` WHERE `travelogue_id`='%s'" % (trav_id)
    qr = "SELECT * FROM `uploads` where upload_type='video' and
`travelogue_id`='%s'"%(trav_id)
    print(qr)
    result = select(qr)
    return jsonify(status="ok", data=result)
```

```
@api.route('/User_view_travel_images', methods=['post'])
def User_view_travel_images():
    data = {}
    trav_id = request.form['travelogueId']
    # qr = "SELECT * FROM `uploads` WHERE `travelogue_id`='%s'" % (trav_id)
    qr = "SELECT * FROM `uploads` where upload_type='image' and
`travelogue_id`='%s'"%(trav_id)
    print(qr)
    result = select(qr)
    return jsonify(status="ok", data=result)
```

```
@api.route('/UserViewOthersTravelogue',methods=['post'])  
def UserViewOthersTravelogue():  
    data = {}  
    loginid=request.form['lid']  
    qr="SELECT * FROM `travelogue` WHERE `login_id` <> '%s' and  
`t_status`='public'"%(loginid)  
    print(qr,"vvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvv")  
    result = select(qr)  
    return jsonify(status="ok", data=result)
```

```
@api.route('/PublicViewOthersTravelogue',methods=['post'])
```

```
def PublicViewOthersTravelogue():
```

```
    data = {}
```

```
    loginid=request.form['lid']
```

```
    qr="SELECT * FROM `travelogue` "
```

```
    result = select(qr)
```

```
    return jsonify(status="ok", data=result)
```

```
@api.route('/PublicviewTravelVideos', methods=['post'])
```

```
def PublicviewTravelVideos():
```

```
    data = {}
```

```
    trav_id = request.form['travelogueId']
```

```
    # qr = "SELECT * FROM `uploads` WHERE `travelogue_id`='%s'" % (trav_id)
```

```
    qr = "SELECT * FROM `uploads` where upload_type='video' and  
`travelogue_id`='%s'"%(trav_id)
```

```
    print(qr)
```

```
    result = select(qr)
```

```
    return jsonify(status="ok", data=result)
```

```
@api.route('/PublicviewTravelImages', methods=['post'])
```

```
def PublicviewTravelImages():
```

```
    data = {}
```

```
    trav_id = request.form['travelogueId']
```

```
    # qr = "SELECT * FROM `uploads` WHERE `travelogue_id`='%s'" % (trav_id)
```

```
    qr = "SELECT * FROM `uploads` where upload_type='image' and  
`travelogue_id`='%s'"%(trav_id)
```

```
    print(qr)
```

```
    result = select(qr)
```

```
    return jsonify(status="ok", data=result)
```



```
@api.route('/UserAddBagdetails',methods=['post'])
def UserAddBagdetails():
    data={}
    loginid=request.form['lid']
    title=request.form['title']
    weight=request.form['weight']
    des=request.form['details']
    # qr_code_filename = "static/qr_code_product/" + str(om_ids)+ ".png"
    # qr_code_data = f"{om_ids}"
    # img = qrcode.make(qr_code_data)
    # img.save(qr_code_filename)

    q= "INSERT INTO `bag_details` VALUES(NULL,(SELECT `user_id` FROM `users`
WHERE `user_id`='%s'),'%s','%s','%s',curdate(),'0')"%(loginid,title,weight,des)
    print(q)
    res=insert(q)
    print(res,"hhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhh")
    # qr="SELECT * FROM bag_details INNER JOIN `users` USING(`user_id`) WHERE
`bag_id`='%s'""%(res)

    qr="SELECT
`users`.`first_name`,`users`.`last_name`,`users`.`place`,`users`.`phone`,`bag_details`.`title`,
bag_details`.`weight`,`bag_details`.`details` FROM bag_details INNER JOIN `users`
USING(`user_id`) WHERE `bag_id`='%s'""%(res)
    myval=select(qr)
    print(type(myval))
```

```

qr_code_filename = "static/qr_code_bag/" + str(res)+ ".png"
qr_code_data = f"{myval}"
print("qrrrrrrrrrrrrrrrrrrrrdata",qr_code_data)
img = qrcode.make(qr_code_data)
img.save(qr_code_filename)
qc="UPDATE `bag_details` SET `qr_code`='%s' WHERE
`bag_id`='%s'"%(qr_code_filename,res)
update(qc)

return jsonify(status="success")

```

```

@api.route('/Viewbag_details_user', methods=['post'])
def details_user():
    data = {}
    qr = "SELECT * FROM `bag_details` INNER JOIN `users` USING(`user_id`)"
    print(qr)
    result = select(qr)
    return jsonify(status="ok", data=result)

```

```

@api.route('/Userbooktour', methods=['post'])
def Userbooktour():
    data = {}
    login_id = request.form['lid']
    package_id = request.form['package_id']
    totalamt = request.form['totalamt']
    date_input = request.form['date']
    ss=date_input.split(' ')
    print(ss, "mncbsdcdcvdsvcsgvcdgsncvdsnccefv")
    quantity = request.form['quantity']
    qr = "SELECT * FROM booking WHERE package_id='%s' AND user_id=(SELECT
user_id FROM users WHERE login_id='%s') AND booking_date='%s'" % (package_id,

```

```

login_id, ss[0])
    res = select(qr)

    if res:
        data['status'] = "duplicate"
        return jsonify(status="duplicate")
    else:
        q = "INSERT INTO booking VALUES(NULL, '%s', (SELECT user_id FROM
users WHERE login_id='%s'), '%s', CURDATE(), '%s', '%s', 'pending')" % (package_id,
login_id, quantity, ss[0], totalamt)
        print(q)
        id = insert(q)
        return jsonify(status="success")

@api.route('/UserViewBookingDetails', methods=['POST'])
def UserViewBookingDetails():
    lid = request.form['lid']
    q = "SELECT * FROM booking INNER JOIN packages USING(package_id) INNER
JOIN hotels using(hotel_id) WHERE user_id=(SELECT user_id FROM users WHERE
login_id='%s')"% (lid)
    print(q, "00000000000000000000000000000000")
    res = select(q)
    print(q)
    return jsonify(status="ok", data=res)

@api.route('/UserMankepayment', methods=['post'])
def UserMankepayment():
    data={}
    amount=request.form['amount']
    book_id=request.form['book_id']
    lid=request.form['lid']
    q="INSERT INTO payment VALUES(null, '%s', '%s', curdate())"% (book_id, amount)
    val=insert(q)
    qr="UPDATE booking SET booking_status='paid' WHERE
booking_id='%s'"%(book_id)
    update(qr)
    return jsonify(status="success")

```

FLUTTER

Main.dart

```
import 'package:flutter/material.dart';
import 'package:fluttertoast/fluttertoast.dart';
import 'package:shared_preferences/shared_preferences.dart';
import 'package:smarttravel/login.dart';
```

```
void main() {
  runApp(const Ambulance());
}
```

```
class Ambulance extends StatefulWidget {
  const Ambulance({super.key});

  @override
  State<Ambulance> createState() => _AmbulanceState();
}
```

```
class _AmbulanceState extends State<Ambulance> {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      theme: ThemeData(primarySwatch: Colors.blue),
      home: const ipset(),
    );
  }
}
```

```
class ipset extends StatefulWidget {
  const ipset({super.key});
```

```

@override
State<ipset> createState() => _ipsetstate();
}

class _ipsetstate extends State<ipset> {
  final TextEditingController ipController = TextEditingController();
  final _formKey = GlobalKey<FormState>(); // Add a global key for the form

  @override
  void initState() {
    super.initState();
    initializeData();
  }

  Future<void> initializeData() async {
    await setIp();
  }

  Future<void> setIp() async {
    SharedPreferences sh = await SharedPreferences.getInstance();
    ipController.text = sh.getString("ipa") ?? "192.168.29.144:5022";
  }

  @override
  Widget build(BuildContext context) {
    return WillPopScope(
      child: Scaffold(
        appBar: AppBar(
          title: const Text("Explore Travels"),
        ),
        body: SafeArea(
          child: Form(
            key: _formKey,
            child: Stack(
              children: <Widget>[
                // Background image
                Image.asset(

```

```

'assets/whi.png',
fit: BoxFit.cover,
width: double.infinity,
height: double.infinity,
),
Center(
  child: Column(
    mainAxisAlignment: MainAxisAlignment.center,
    crossAxisAlignment: CrossAxisAlignment.center,
    children: [
      Padding(
        padding: const EdgeInsets.all(2),
        child: TextFormField(
          controller: ipController,
          decoration: const InputDecoration(
            border: OutlineInputBorder(),
            labelText: "IP Address",
            hintText: "Enter a valid IP address",
          ),
          validator: (value) {
            if (value!.isEmpty) {
              return 'Please enter the IP';
            }
            return null; // Return null if the input is valid
          },
        ),
      ),
      Padding(
        padding: const EdgeInsets.all(16.0),
        child: ElevatedButton(
          onPressed: () async {
            String ipaddress = ipController.text.toString();
            if (ipaddress.isEmpty) {
              Fluttertoast.showToast(
                msg: "Enter A valid Ip Address To login",
                toastLength: Toast.LENGTH_SHORT,
                gravity: ToastGravity.CENTER,

```

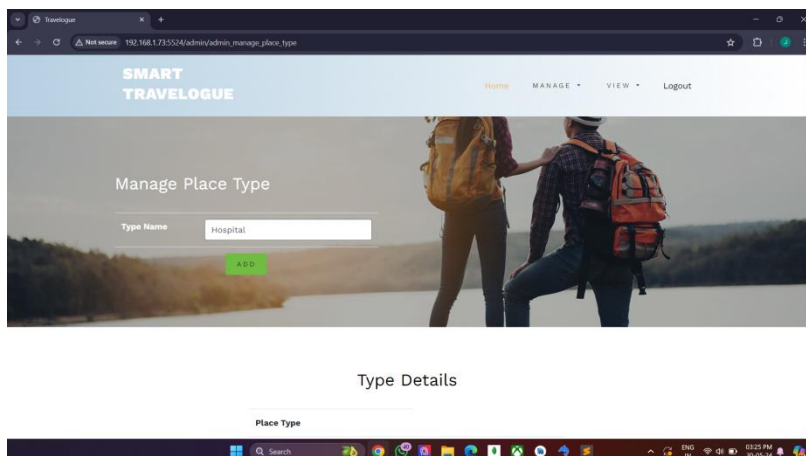
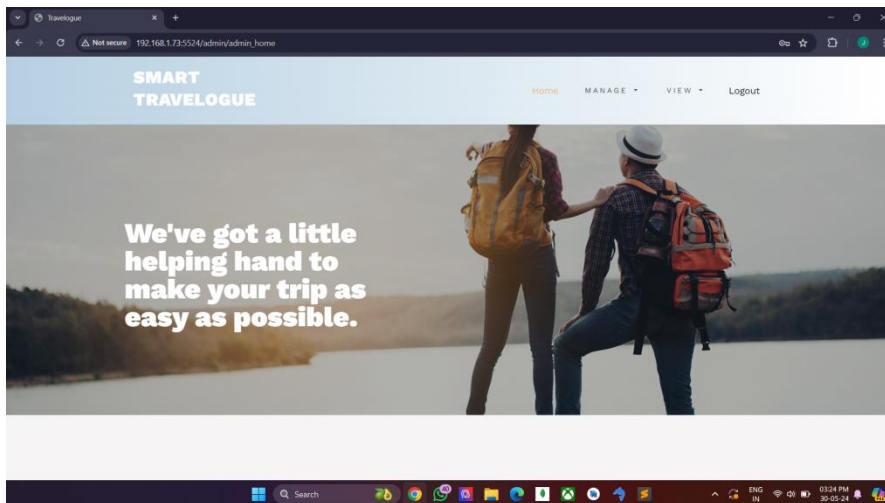
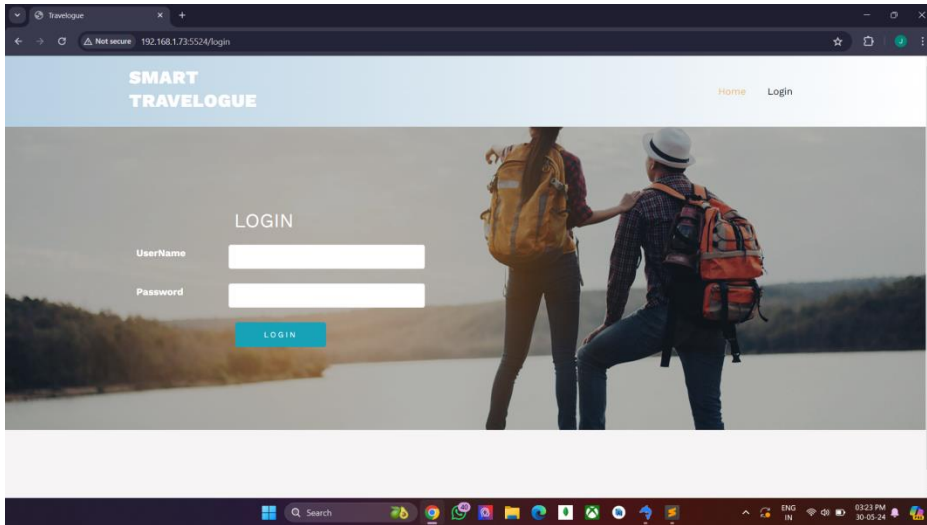
```

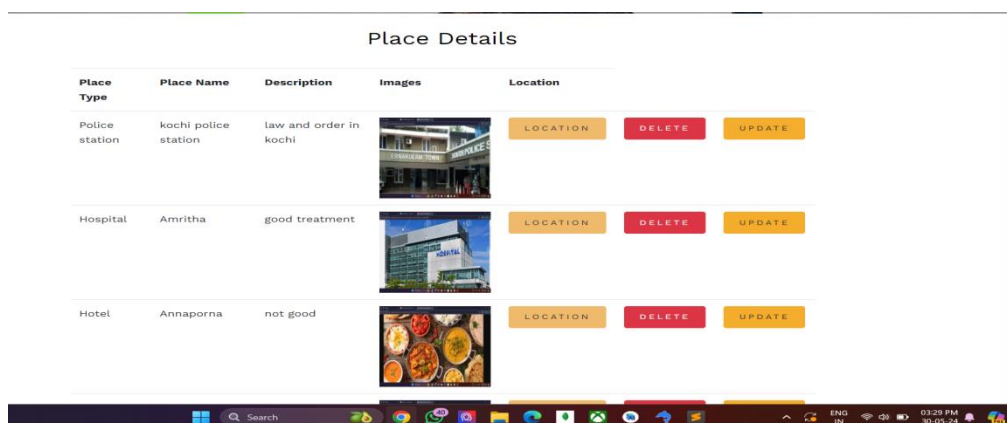
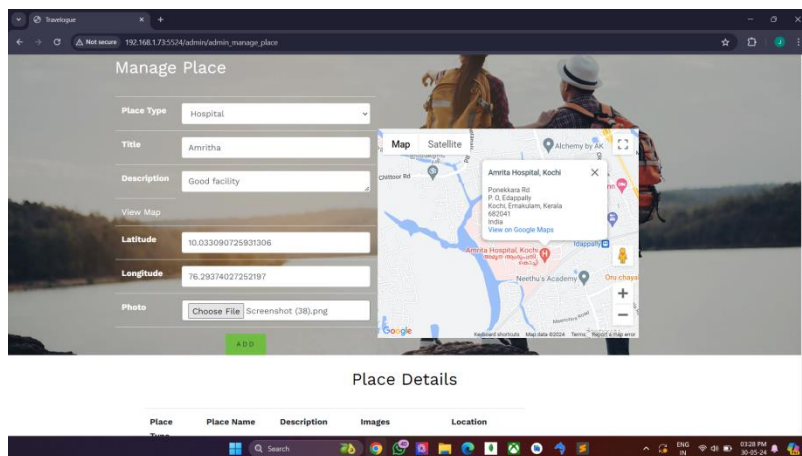
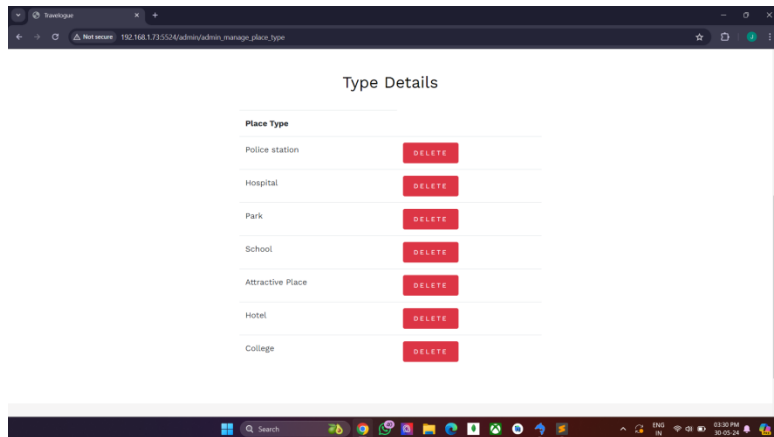
        timeInSecForIosWeb: 1,
        backgroundColor: Colors.red,
        textColor: Colors.white,
        fontSize: 16.0,
      );
    } else {
      String url = "http://" + ipController.text.toString();
      SharedPreferences sh =
        await SharedPreferences.getInstance();
      sh.setString("ip", url);
      sh.setString("ipa", ipaddress);
      print("Entered IP Address: $ipaddress");
      Navigator.push(
        context,
        MaterialPageRoute(builder: (context) => login()),
      );
    }
  },
  child: const Icon(Icons.key),
),
),
],
),
),
],
),
),
),
),
onWillPop: () async {
  Navigator.of(context).pop(true);
  return true;
},
);
}
}

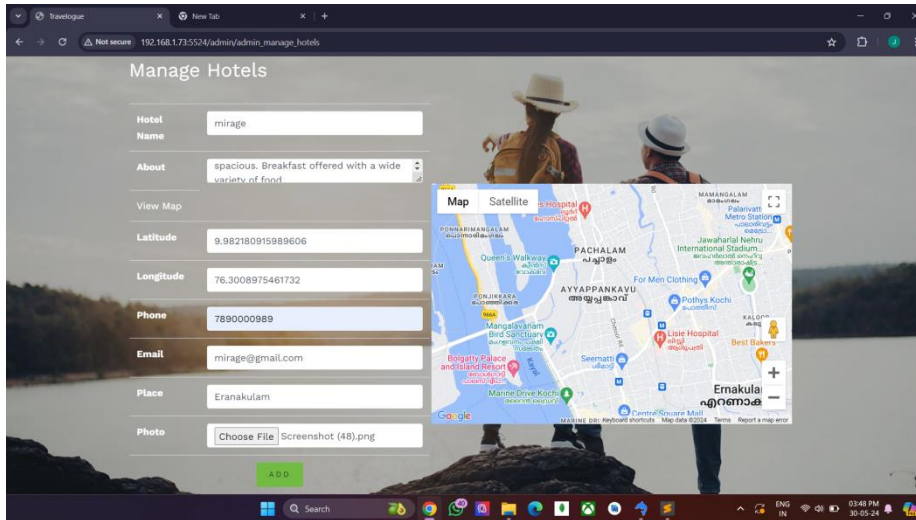
```

.

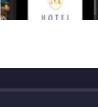
GLOSSARY

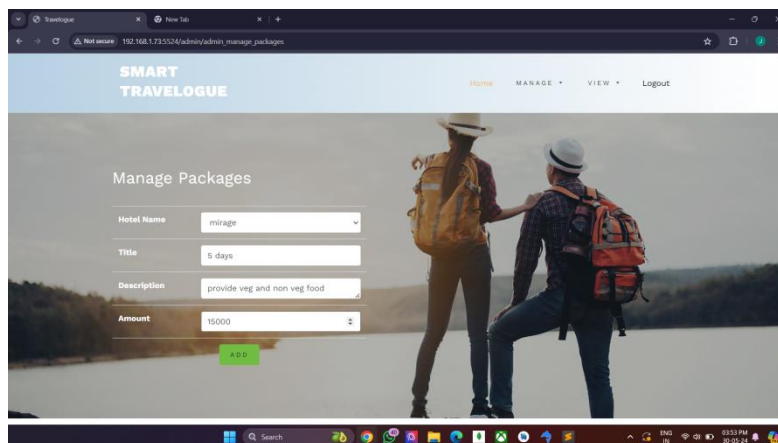


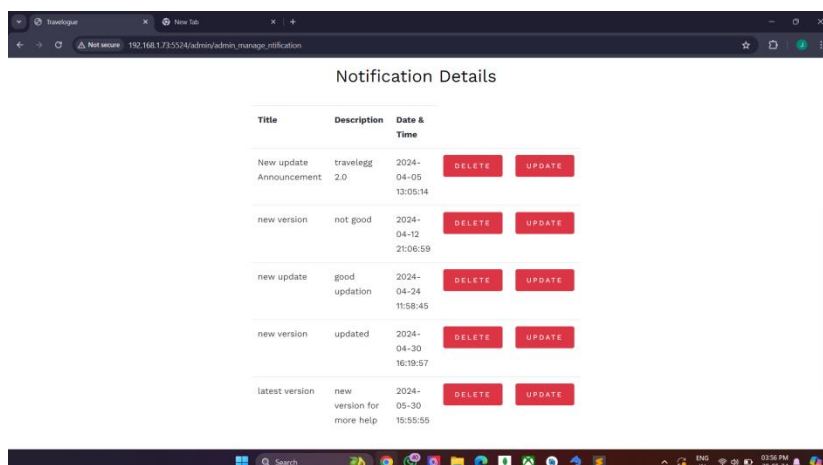
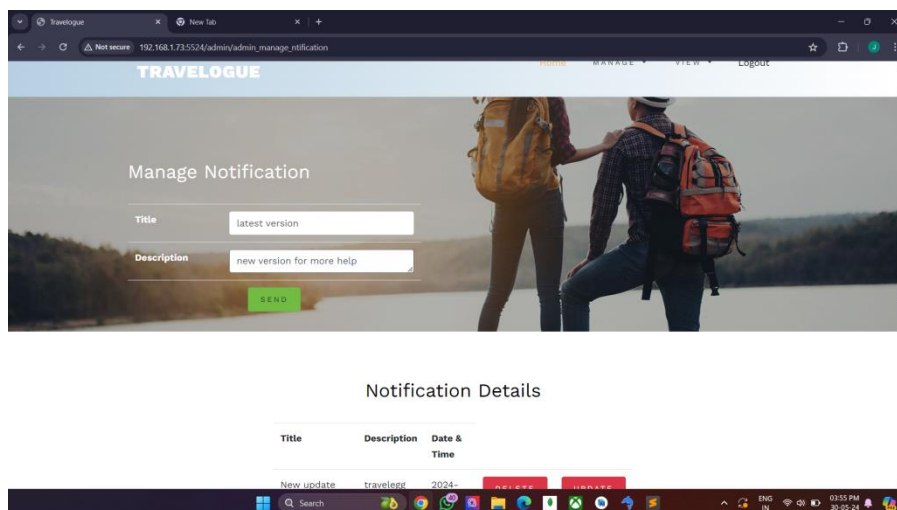
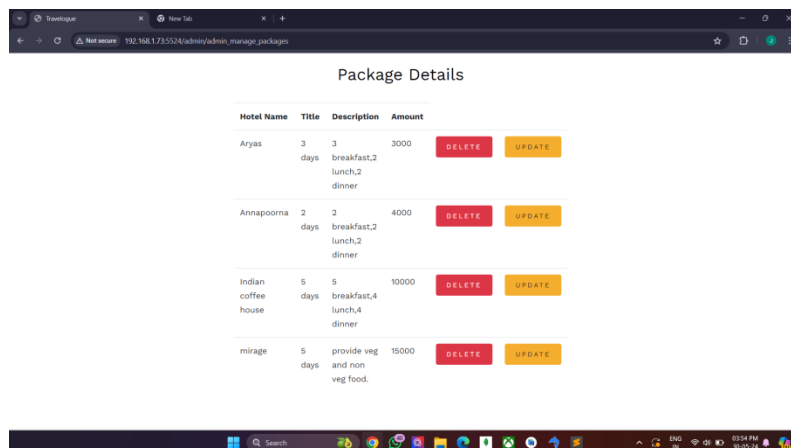




Details

Hotel Name	About	Phone	Email	Photo	Place	
Aryas	good veg food	9000000089	aryas@gmail.com		palakad	DELETE UPDATE
Annapoorna	not good	78900009898	annaporana@gmail.com		Alappuzha	DELETE UPDATE
Indian coffee house	good food	7890000989	coffeehouse@gmail.com		Alappuzha	DELETE UPDATE
mirage	the hotel room was clean,nice	7890000989	mirage@gmail.com		Ernakulam	DELETE UPDATE





SMART TRAVELOGUE

[Home](#) [MANAGE](#) [VIEW](#) [Logout](#)

User Details

User Name	House Name	Place	Phone	Email
Abhijith Aijith	Rdx,kurtanad,mambuzhakary	Alappuzha	7474748999	abhi@gmail.com
Abel Tom	abel villa	Kottayam	8488574749	abel@gmail.com
nisha jacob	nest	chengannur	7484848480	nish@gmail.com
lekshmi priya	lekshmi bhavan	Chengannur	9876543467	lekshmi@gmail.com

SMART TRAVELOGUE

[Home](#) [MANAGE](#) [VIEW](#) [Logout](#)

View Travelogues

User	Place	Title	Description	Date	Action
Abhijith Aijith	KASHMIR	Heaven	Best place to visit	2024-04-05 12:41:44	public Select
Abel Tom	Goa	VIBE PLACE	the best experience	2024-04-05 13:02:21	APPROVE Select
nisha jacob	IDUKKI	beautiful place	one of the natural wonders of kerala	2024-04-05 13:03:58	public Select
Abel Tom	alappuzha	venice of india	best place for boating	2024-04-12 21:41:12	public Select
lekshmi priya	Africa	village of africa	good	2024-04-23 16:02:59	public Select

SMART TRAVELOGUE

[Home](#) [MANAGE](#) [VIEW](#) [Logout](#)

Feedback

User Name	Description	Date
Abhijith Aijith	superrrr	2024-04-05 12:42:23
Abel Tom	love ittt	2024-04-05 13:03:05
nisha jacob	nice	2024-04-05 13:04:17
Abel Tom	good	2024-04-24 13:10:02
lekshmi priya	love it	

Flutter

